NORTHWESTERN UNIVERSITY

Contract Checking as Effects

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

Master of Science

Field of Computer Science

By

Anh Khoa Nguyen

EVANSTON, ILLINOIS

December 2026

Copyright by Anh Khoa Nguyen 2026

All Rights Reserved

ABSTRACT

Contract Checking as Effects

Anh Khoa Nguyen

nice

Acknowledgements

Text for acknowledgments goes here. You can thank your advisors, peers, and family.

Preface

This is the preface (optional).

Dedication

This is the dedication (optional).

Table of Contents

ABSTRACT **3**
Acknowledgements **4**
Preface **5**
Dedication **6**
Table of Contents **7**
List of Tables **9**
List of Figures **10**
1 Introduction **11**
1.1 Contributions 12
1.2 Base Language 13
2 Contracts Language **20**
2.1 Example 20
2.2 Language Model 21
3 Effects Language **28**
3.1 Example 28
3.2 Language Model 29
4 Formal Compiler **37**
4.1 Handler design 38
4.2 Effects annotation 39
4.3 Compiler Type Preservation and Effect Safety 39
4.4 Compiler Correctness 39
4.5 Compiling Contracts for Mutable Cells 42
4.5.1 Observation 42
4.5.2 Unified Interface 42
5 Implementation **45**

5.1 Contract States and Interaction 47
5.1.1 Global State 47
5.2 Contract Interaction 50
Bibliography 51
A Base Proofs 53
A.1 Type Safety 53
B Contracts Theorems 56
C Contract Erasure Theorem 61
C.1 Relation 61
C.2 Main Theorem Proof 65
C.3 Conventions 70
C.4 Evaluation Contexts 71
C.5 Statements and proofs 72
D Compiler Typesoundness Proof 83
D.1 Overview 83
D.2 Type Translation 83
D.3 Compiler Type Preservation 84
D.4 Wrapped Compiled Program Typing 86
E Compiler Correctness Proof (Deep Handler) 88
E.1 Overview 88
E.2 Relation 88
E.3 Compiler Correctness 94
F Compiler Correctness Proof (Shallow Handler) 102
F.1 Overview 102
F.2 Relation 103
F.3 Compiler Correctness 106

List of Tables

List of Figures

Figure 1 Base Language 14
Figure 2 Type Rules for Base 16
Figure 3 Reduction Rules for Base 18
Figure 4 Contracts in action 20
Figure 5 Contracts language 22
Figure 6 Type Rules for Contracts 24
Figure 7 Reduction Rules for Contracts 26
Figure 8 Handler in action 28
Figure 9 Handler with state 29
Figure 10 Type Rules for Effects 33
Figure 11 Compiler Rules 41
Figure 12 Structure of the contract erasure proof. Dashed vertical lines denote ER. 65
Figure 13 Typing of evaluation contexts. The frames zero?(E), $\langle E, e\rangle$, $\langle v, E\rangle$, inj.l(E), inj.r(E) have rules CE-Zero, CE-Tuple1, CE-Tuple2, CE-Inj-Left, CE-Inj-Right of the same form, following the type rule of the frame. 72
Figure 14 Structure of the compiler correctness proof. Dashed lines rel states by EQ. 95

CHAPTER 1

Introduction

Software contracts, or *behavioral contracts*, allow programmers to attach assertions to values, and functions. These assertions are runtime guarantees, on violation, an error is thrown, blaming the violating party. Contracts are most useful when placed on a function, enforcing the function's input and output values. While first-order functional contracts are useful, they are too simple. Higher-order functional contracts allows programmers to enforce contracts for higher-order functions. Dependent contracts are functional contracts allowing usage of the function argument during post-condition contract checking.

Recently, algebraic effect handlers has gained a large attraction, and is adopted in OCaml since version 5.0. Effect handlers open a new way to identify effects and handling them. Handlers are defined to handle one or many effects. A handler for an effect can stop the program or continue with some result. Handlers do not have state by default, but it can be implemented by applying the new states to the continuation closure, and let the underlying computation receives a state argument.

Combining the contract system and effect handlers is a new approach. Effectful Contract [11] demonstrates how both systems can be combined, providing contracts for effectful code. Contracts in their system remain a language feature. In this dissertation, instead of combining the effects and contracts, we ask ourselves whether contract checking itself can become an effect, and reuse the capabilities of handlers.

> *Contract checking can be encoded as effects, and delegate them to handlers can unify all contracts in the same context, while separated from main code.*

1.1 Contributions

This dissertation provides concrete evidence to the thesis claim. We model the two languages Contracts representing software contracts, and `Effects` representing effect handlers. Then a formal model for a compiler between Contracts and `Effects` is presented. This compiler is proven correct, and effect safe. Moreover, we provide an implementation of such compiler by providing a preprocessor to convert OCaml code annotated with contracts into run-able OCaml code based off this compiler model. Lastly, we provide a Redex PLT model to test our theorems.

Structure ...

1.2 Base Language

We define Base language and its syntax in Figure 1. The language extends the PCF language with tuples. It has no mutable cells. This will be the base language for both Contracts and `Effects`.

Base

$$
\begin{array}{rcl}
\tau & ::= & o \\
& | & \tau \to \tau \\
& | & \langle \tau, \tau \rangle \\
& | & \mathsf{unit} \\
\\
o & ::= & \mathrm{num} \\
& | & \mathrm{bool} \\
\\
e & ::= & n \mid b \mid \mathsf{unit} \mid \langle e, e \rangle \mid x \\
& | & \lambda x : \tau . e \mid \mu x : \tau . e \\
& | & e + e \mid e - e \mid e \wedge e \mid e \vee e \\
& | & e\, e \mid \mathrm{if}\ e\ \mathrm{then}\ e\ \mathrm{else}\ e \\
& | & \mathsf{zero?}(e) \\
& | & \mathsf{inj.l}(e) \mid \mathsf{inj.r}(e)
\end{array}
$$

$$
\begin{array}{rcl}
n & ::= & 0 \mid -1 \mid 1 \mid \ldots \\
\\
b & ::= & \mathsf{true} \mid \mathsf{false} \\
\\
v & ::= & n \mid b \mid \mathsf{unit} \mid \langle v, v \rangle \mid x \mid \lambda x : \tau . e \\
\\
E & ::= & \square \\
& | & E + e \mid v + E \mid E - e \mid v - E \\
& | & E \wedge e \mid v \wedge E \mid E \vee e \mid v \vee E \\
& | & E\, e \mid v\, E \mid \mathrm{if}\ E\ \mathrm{then}\ e\ \mathrm{else}\ e \mid \mathsf{zero?}(E) \\
& | & \langle E, e \rangle \mid \langle v, E \rangle, \mathsf{inj.l}(E) \mid \mathsf{inj.r}(E)
\end{array}
$$

Figure 1: Base Language

$$\frac{}{\Gamma \vdash n : \mathsf{num}}\ \text{T-Num} \qquad \frac{}{\Gamma \vdash b : \mathsf{bool}}\ \text{T-Bool} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle}\ \text{T-Tuple}$$

$$\frac{}{\Gamma \vdash \mathsf{unit} : \mathsf{unit}}\ \text{T-Unit} \qquad \frac{o : \tau_a \times \tau_b \to \tau_o \quad \Gamma \vdash e_1 : \tau_a \quad \Gamma \vdash e_2 : \tau_b}{\Gamma \vdash e_1 o e_2 : \tau_o}\ \text{T-Op}$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1 . e : \tau_1 \to \tau_2}\ \text{T-Lambda} \qquad \frac{\Gamma, x : \tau \vdash e : \tau}{\Gamma \vdash \mu x : \tau . e : \tau}\ \text{T-Fix}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \text{T-App} \qquad \frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash e : \mathsf{num}}{\Gamma \vdash \mathsf{zero?}(e) : \mathsf{bool}}\ \text{T-Zero}$$

$$\frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.l}(e) : \tau_1}\ \text{T-Inj-Left} \qquad \frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.r}(e) : \tau_2}\ \text{T-Inj-Right}$$

Figure 2: Type Rules for Base

$\lambda x.e\ v$	$\dashrightarrow$	$e[x := v]$	S-App
$\mu x.e$	$\dashrightarrow$	$e[x := \mu x.e]$	S-Fix
if true then e_1 else e_2	$\dashrightarrow$	e_1	S-If-True
if false then e_1 else e_2	$\dashrightarrow$	e_2	S-If-False
$\mathsf{inj.l}(\langle v_1, v_2 \rangle)$	$\dashrightarrow$	v_1	S-Inj-Left
$\mathsf{inj.r}(\langle v_1, v_2 \rangle)$	$\dashrightarrow$	v_2	S-Inj-Right
$v_1 o v_2$	$\dashrightarrow$	$\delta(o, v_1, v_2)$	S-Op
$\mathsf{zero?}(n)$	$\dashrightarrow$	b	S-Zero

$$\frac{e_1 \dashrightarrow e_2}{E[e_1] \longrightarrow E[e_2]}\ \text{Step}$$

Figure 3: Reduction Rules for Base

CHAPTER 2

Contracts Language

2.1 Example

Contracts provide a run-time validation of programs, similar to how assertions work. Contracts extend basic assertions to support functions, especially higher-order ones, correctness. Past research has move from pre/post condition placing on functions, into obligations [5], or monitors. These constructions support blame tracking, enabling the developer to know *who*, which function, is responsible for a violation.

Figure 4 illustrates a function protected with a monitor. This function should return a value that is even. When applied with 10, it is checked by the function's *domain contract*. Since the function does not impose any properties for its argument, the contract passes. The body of the function is executed normally, but its result must pass the function's *range contract*. Because 11 is not an even number, the program terminates with a blame.

$$
\begin{array}{ll}
 & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\to & \mathsf{guard}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{mon}_j^{l,k}(\mathsf{flat}(\lambda x.\ \mathsf{true}), 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{guard}_j^{l,k}(\lambda x.\ \mathsf{true}, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}((\lambda x.\ \mathsf{true})\ 10, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}(\mathsf{true}, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ 10) \\
\to^* & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), 11) \\
\to^* & \mathsf{blame}_j^{k}
\end{array}
$$

Figure 4: Contracts in action

2.2 Language Model

We define Contracts, in Figure 5, by extending the Base language with contracts. Contracts protect an expression, ensuring that the evaluated value conform to the contracts defined. Contracts are defined for all possible types of value. Base values, including booleans and numbers, are trivial to enfore a contract on them. While functions need to check for both its argument and return result. Not only that, functions are first-class citizen in the language, and we can form higher-order functions. A contract for function may want to "see" the argument when checking the result, such is the idea of dependent contracts. Both contracts higher-order functions, and dependent contracts semantics are discussed in [5].

The language supports contracts for tuples. Their contracts will be applied individually during execution. Mutable cells contracts follow the work of [2], where a special contract type ref/c(κ) is used to protect the cell's value.

Contracts extends Base

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
& | & \mathsf{con}(\tau) \\
\\
e & ::= & \ldots \\
& | & \mathsf{mon}_j^{k,l}(\kappa, e) \\
& | & \mathsf{check}_j^k(e, v) \\
& | & \mathsf{blame}_p^l \\
\end{array}
\qquad
\begin{array}{rcl}
\kappa & ::= & \mathsf{flat}(e) \mid \kappa \to \kappa \mid \kappa \to_i \lambda x : \tau.\kappa \\
& | & \langle \kappa, \kappa \rangle \mid \mathsf{ref/c}(\kappa) \\
\\
v & ::= & \ldots \\
& | & \mathsf{guard}_j^{k,l}(\kappa, v) \\
\\
E & ::= & \ldots \\
& | & \mathsf{mon}_j^{k,l}(\kappa, E) \\
& | & \mathsf{check}_j^k(E, v)
\end{array}
$$

Figure 5: Contracts language

$$\frac{\Gamma \vdash e : \tau \to \mathsf{bool}}{\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)}\ \text{Type-C-Flat} \qquad \frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \quad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-Fun}$$

$$\frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \quad \Gamma \vdash \lambda x : \tau_1 . \kappa_2 : \tau_1 \to \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to_i \lambda x . \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-DepFun}$$

$$\frac{\Gamma \vdash \kappa : \mathsf{con}(\tau)}{\Gamma \vdash \mathsf{ref/c}(\kappa) : \mathsf{con}(\mathsf{ref}(\tau))}\ \text{Type-C-Ref}$$

$$\frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \quad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \langle \kappa_1, \kappa_2 \rangle : \mathsf{con}(\langle \tau_1, \tau_2 \rangle)}\ \text{Type-C-Tuple}$$

$$\frac{\Gamma \vdash \kappa : \mathsf{con}(\tau) \quad \Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau}\ \text{Type-C-Mon} \qquad \frac{}{\Gamma \vdash \mathsf{blame}_l^k : \tau}\ \text{Type-C-Blame}$$

Figure 6: Type Rules for Contracts

When a contract violation occurs, a blame is thrown with the party, caller or callee. The semantics for correct blaming has been studied under [2] and [3]. Following previous works, We use *indy* semantics for dependent contracts. We use $\mathsf{guard}_j^{k,l}(\kappa, v)$ as the run-time proxy to value v procted under a contract κ. Guard is a value, the semantics adapt to usage of guards, and expand further, sometimes back to monitor to evaluate inner expressions. A guard on a basic value will trigger a $\mathsf{check}_j^k(e\ v, v)$ and returns the value v if it passes the predicate e, or returns a blame_j^k to party k with contract location j. The evaluation rules for Contracts is presented in Figure 7.

$$\mathsf{check}_j^k(\mathrm{tt}, v) \dashrightarrow v \qquad \text{Check-True}$$

$$\mathsf{check}_j^k(\mathrm{ff}, v) \dashrightarrow \mathsf{blame}_j^k \qquad \text{Check-False}$$

$$\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) \qquad \text{Mon-Flat}$$

$$\mathsf{mon}_j^{k,l}(\kappa, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa, v) \qquad \text{Mon-Guard}$$

$$\text{where } \kappa \neq \mathsf{flat}(e)$$

$$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big) \qquad \text{Guard-Func}$$

$$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\Big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big) \qquad \text{Guard-Dep}$$

$$\text{where } \kappa_3 = \Big\{\mathsf{mon}_j^{l,j}(\kappa_1, v_2)/x\Big\}\kappa_2$$

$$\mathsf{get}\Big(\mathsf{guard}_j^{k,l}(\kappa, v)\Big) \dashrightarrow \mathsf{mon}_j^{k,l}(\kappa, \mathsf{get}(v)) \qquad \text{Mon-Get}$$

$$\mathsf{set}\Big(\mathsf{guard}_j^{k,l}(\kappa, v_1), v_2\Big) \dashrightarrow \mathsf{mon}_j^{k,l}(\mathsf{ref/c}(\kappa), e) \qquad \text{Mon-Set}$$

$$\text{where } e = \mathsf{set}\Big(v_1, \mathsf{mon}_j^{l,k}(\kappa, v_2)\Big)$$

$$\frac{E \neq \square}{E\big[\mathsf{blame}_j^k\big], \sigma \to \mathsf{blame}_j^k, \sigma} \text{ Step-C-Blame} \qquad \frac{e_1 \dashrightarrow e_2}{E[e_1], \sigma \to E[e_2], \sigma} \text{ Step-C}$$

Figure 7: Reduction Rules for Contracts

CHAPTER 3

Effects Language

3.1 Example

In Figure 8, we present a simple example: a program wants to compute $(\mathscr{D}\ 2) + 1$ is registered with a handler h for effect $\mathscr{D}$, basically *duplicating* its input. When an effect is perform, the main program pauses the execution, and push everything later into a continuation $k = \lambda y.\ \texttt{handle}\ h\ (y + 1)$, at (i). Here, the continuation register the handler h again, in case the rest of the program uses $\mathscr{D}$. When the program is a value, the handler removes itself (ii).

$$
\begin{array}{rll}
h & = & \{\mathscr{D} \to \lambda v.\lambda k.k\ (v + v)\} \\
 & \texttt{handle}\ h\ ((\texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2) + 1) & \\
\Longrightarrow & \texttt{handle}\ h\ \ \texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2 & \\
\Longrightarrow & (\lambda v.\lambda k.k\ (v + v))\ 2\ (\lambda y.\ \texttt{handle}\ h\ (y + 1)) & \text{(i)} \\
\Longrightarrow^{*} & (\lambda y.\ \texttt{handle}\ h\ (y + 1))\ (2 + 2) & \\
\Longrightarrow^{*} & \texttt{handle}\ h\ 5 & \text{(ii)} \\
\Longrightarrow & 5 &
\end{array}
$$

Figure 8: Handler in action

Handlers can encode states. This is a nice feature to provide a common state across the code. We illustrate the example in Figure 9, where a simple state can be managed by two effects *get* $\mathscr{G}$, and *set* $\mathscr{S}$. As can be seen, the continuation now returns a function expecting a state. During handler execution, this state is applied automatically if the handler is installed with initial state. The continuation resumes the program with the new state, made possible because handler is installed with new state.

Without careful setup, the program might enter the stuck state because the handler removes itself when the execution completes, making the result applies with the state. For example, this program $\texttt{handle}\ h\ v\ s \Longrightarrow^* v\ s$ stucks if v is not callable. There are many ways to resolve this problem. The first approach requires the program awareness of the state. The programmer explicitly returns a function taking the result and the state but only returns the result.

$$\texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ e)\ s \Longrightarrow^* \texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ v)\ s \Longrightarrow^* (\lambda s.v)\ s \Longrightarrow v$$

The second approach is to use *parameterized handlers* [8], which puts the state into the handler expression. The third approach is to augment the handler with a special effect `return`, which modifies the value before removing the handler. The example Figure 8 chooses the first approach, as it keeps the handler semantics simple, although a programmer needs to pay more attention when dealing with handlers receiving arguments.

$$h \quad = \quad \{\mathscr{G} \to \lambda v.\lambda k.\lambda s.k\ s\ s; \mathscr{S} \to \lambda v.\lambda k.\lambda s.k\ s\ v\}$$

$$\begin{aligned}
&(\texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ ((\texttt{perform}\ \mathscr{S}\ \mathsf{num}\ 42) + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + \\
&100))) \ 0 \\
\Longrightarrow^* \ & (\lambda v.\lambda k.\lambda s.k\ s\ v)\ 42\ \lambda y.\ \texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100))\ 0 \\
\Longrightarrow^* \ & (\lambda y.\ \texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100)))\ 0\ 42 \\
\Longrightarrow^* \ & \texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ ((0 + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100)))\ 42 \\
\Longrightarrow^* \ & \texttt{handle}\ h\ ((\lambda x.\lambda s.x)\ 142)\ 42 \\
\Longrightarrow^* \ & (\lambda s.142)\ 42 \\
\Longrightarrow^* \ & 142
\end{aligned}$$

Figure 9: Handler with state

3.2 Language Model

`Effects`, inspired by [16], equips Base with effects and effect handlers. We extend the type system into a kinded type system based on System F, mainly to separate between effect types and normal types. System F also provides polymorphism. The type system

adapts its syntax of functions into $\tau_1 \to^{\varepsilon} \tau_2$ denotes an effect ε (may) occurs during the execution of the function.

Following [16], effects are group of "operations", distinct by its label. An operation $\mathcal{O}$ of effect ε is invoked by applying `perform` $\mathcal{O}\ \tau$ with an argument, and produces an effect ε. Given an expression e that would perform some effect ε, a handler h providing all definitions for operations of ε must be installed. The calculus provides two ways to install a handler, `handle` $h\ e$ and `handler` $h\ (\lambda_.e)$. The former is the base syntax, during evaluation of e, the handler h is used. The other syntax is useful for suspending a computation, and run it only when a handler is applied, providing flexibility such as different handlers.

A handler is a mapping between operation names and their definitions. An operation is a function with the template $\Lambda\alpha.\lambda x : \tau_1.\lambda k : \tau_2.e$. The polymorphic type variable α is used to make operations polymorphic. The argument x is the operation argument. The argument k is the *continuation* or *resumption*, enclosing the rest of the program after the operation invocation. When k is called, the program resumes with the argument provided to k.

Because k is a function, a handler can return k and let the program use continuation anyhow the programer wants. This is described in [16] as "undesirable in practice" and define a restriction named *scoped resumption*. Our language also follows the same principle and does not allow returning the continuation k (directly or indirectly through closure). However, we do not enforce this and left out for the language implementators.

`Effects` extends Base

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
& | & \alpha^k \mid c^k \tau \ldots \mid \tau \to^\varepsilon \tau \mid \forall \alpha^k . \tau \\
& | & \texttt{blab} \\
\\
k & ::= & * \\
& | & k \to k \\
& | & \texttt{eff} \\
& | & \texttt{lab} \\
\\
e & ::= & \ldots \\
& | & \mathsf{lambda}^\varepsilon x : \tau . e \\
& | & e[\tau] \\
& | & \texttt{handle}\ h\ e^\varepsilon \\
& | & \texttt{error}_p \\
\\
h & ::= & \{\mathsf{op} \to f, \ldots, \} \\
\\
f & ::= & \Lambda \alpha . \mathsf{lambda}^\varepsilon x : \tau_1 . \mathsf{lambda}^\varepsilon k : (\tau_2 \to^\varepsilon \tau) . e
\end{array}
$$

$$
\begin{array}{rcl}
v & ::= & \ldots \\
& | & \mathsf{lambda}^\varepsilon x : \tau . e \\
& | & \Lambda \alpha^k . v \\
& | & \texttt{handler}\ h^\varepsilon \\
& | & \texttt{perform}\ \varepsilon\ \mathsf{op}\ \tau \\
& | & \texttt{error}_p \\
\\
p & ::= & \text{Blame Party} \\
\\
E & ::= & \ldots \\
& | & E[\tau] \\
& | & \texttt{handle}^{\varepsilon(h,E)} \\
& | & \texttt{error}_E \\
\\
F & ::= & \square \\
& | & F + e \mid v + F \mid F - e \mid v - F \\
& | & F \wedge e \mid v \wedge F \mid F \vee e \mid v \vee F \\
& | & F\ e \mid v\ F \\
& | & \text{if } F \text{ then } e \text{ else } e \\
& | & F[\tau]
\end{array}
$$

Listing 1: `Effects` language

A function can use multiple effects, and are represented as rows with equivalence up to reordering. An empty effect is represented as $\langle\rangle$. Effects can be concatenated by $\langle l, \varepsilon\rangle$ where effect row ε is extended with effect l. For ease of writing, we allow writing $\langle l_1, l_2, ..., l_n\rangle$ as shorthand instead of $\langle l_1, \langle l_2, ...\langle l_n\rangle\rangle\rangle$.

The language is explicit about the effects. Lambdas are annotated with effects, as well as perform (operation invocation), and handler installation. These effect annotations are used only during type checking, they do not affect the execution.

Typing for an effect language requires some consideration. Following the works in [16], we define the type judgement relationship $\Delta \vdash e : \tau \mid \varepsilon$ between a type context Δ, an expression e and assert it against type τ with some possible effects ε. Variables and base values do not have effects, they can take any effects. For this reason, we define $\Delta \vdash_{\text{val}} v : \tau$. Handlers are typed using $\Delta \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$, that it handles a *single* effect l, other effects in ε remains unhandled, and all operations returns τ. All effects signatures are defined in a global set Σ for fast accessing the type of the operations. This set can be built by going through all handlers, and its existence is assumed during the start of type checking. All type rules unique to `Effects` is presented in Figure 10.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash_{\text{val}} x : \tau}\ \text{T-E-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid \varepsilon}{\Gamma \vdash_{\text{val}} \mathsf{lambda}^{\varepsilon} x : \tau_1 . e : \tau_1 \to^{\varepsilon} \tau_2}\ \text{T-E-Abs}$$

$$\frac{\Gamma \vdash_{\text{val}} v : \tau}{\Gamma \vdash v : \tau \mid \varepsilon}\ \text{T-E-Val} \qquad \frac{k \neq \mathtt{lab} \quad \Gamma \vdash_{\text{val}} v : \tau}{\Gamma \vdash_{\text{val}} \Lambda\alpha^{k}.v : \forall\alpha^{k}.\tau}\ \text{T-E-TAbs}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to^{\varepsilon} \tau_2 \mid \varepsilon \quad \Gamma \vdash e_2 : \tau_1 \mid \varepsilon}{\Gamma \vdash e_1\ e_2 : \tau \mid \varepsilon}\ \text{T-E-App} \qquad \frac{\vdash_{\text{wf}} \tau : k \quad \Gamma \vdash e : \forall\alpha^{k}.\tau_1 \mid \varepsilon}{\Gamma \vdash e[\tau] : \tau_1[\alpha := \tau] \mid \varepsilon}\ \text{T-E-Tapp}$$

$$\frac{\mathsf{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l) \qquad \alpha \notin \text{ftv}(\Gamma)}{\Gamma \vdash_{\text{val}} \mathtt{perform}^{\varepsilon}\ \mathsf{op}\ \tau : \tau_1[\alpha := \tau] \to^{\langle l \mid \varepsilon \rangle} \tau_2[\alpha := \tau]}\ \text{T-E-Perform}$$

$$\frac{\mathsf{op}_i : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l) \qquad \alpha \notin \text{ftv}(\varepsilon, \tau) \quad \Gamma \vdash_{\text{val}} f_i : \forall\alpha.\tau_1 \to^{\varepsilon} ((\tau_2 \to^{\varepsilon} \tau) \to^{\varepsilon} \tau)}{\Gamma \vdash_{\text{ops}} \{\mathsf{op}_1 \to f_1, \ldots, \mathsf{op}_n \to f_n\} : \tau \mid l \mid \varepsilon}\ \text{T-E-Ops}$$

$$\frac{\Gamma \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon}{\Gamma \vdash_{\text{val}} \mathtt{handler}^{\varepsilon(h)} : (\mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau) \to^{\varepsilon} \tau}\ \text{T-E-Handler}$$

$$\frac{\Gamma \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon \quad \Gamma \vdash e : \tau \mid \langle l \mid \varepsilon \rangle}{\Gamma \vdash \mathtt{handle}^{\varepsilon(h,e)} : \tau \mid \varepsilon}\ \text{T-E-Handle}$$

Figure 10: Type Rules for `Effects`

Lastly the semantic model of `Effects` is presented in Listing 2. A handler can be installed to a function as a shorthand, where it applies the function with a unit argument, which to be ignored. The reduction for handling an effect calling is encoded as *deep handler*. Supporting the shallow should be straightforward by not installing the handler again in the continuation k. There is little advantage to support shallow handlers, therefore, only deep handlers are used. While looking for the handler, we make sure we get the innermost handler that can handle the operation.

$$(\Lambda\alpha^{k}.v)[\tau] \quad \rightarrow \quad v[\alpha := \tau] \qquad \text{Step-E-TApp}$$

$$\texttt{handler}^{\varepsilon(h)}\ v \quad \rightarrow \quad \texttt{handle}^{\varepsilon(h,(v\ ()))} \qquad \text{Step-E-Handler}$$

$$\texttt{handle}^{\varepsilon(h,v)} \quad \rightarrow \quad v \qquad \text{Step-E-Return}$$

$$\frac{\mathsf{op} \notin \texttt{bop}(E) \wedge (\mathsf{op} \rightarrow f) \in h \qquad \mathsf{op} : \forall\alpha.\tau_1 \rightarrow \tau_2 \in \Sigma(l) \qquad k = \mathsf{lambda}^{\varepsilon} x : \tau_2[\alpha := \tau].\texttt{handle}^{\varepsilon(h,E[x])}}{\texttt{handle}^{\varepsilon\left(h,E\left[\texttt{perform}^{\varepsilon}_{p(\mathsf{op},\tau:\tau)}\ v\right]\right)} \quad \rightarrow \quad f[\tau]\ v\ k} \text{ Step-E-Perform}$$

$$\frac{E \neq \square}{E\left[\texttt{error}_p\right], \sigma \Longrightarrow \texttt{error}_p, \sigma} \text{ Step-E-Error} \qquad \frac{e_1 \rightarrow e_2}{E[e_1], \sigma \Longrightarrow E[e_2], \sigma} \text{ Step-E}$$

Helper functions

$$\begin{array}{lcl}
\texttt{bop}(\square) & = & \varnothing \\
\texttt{bop}(E\ e) & = & \texttt{bop}(E) \\
\vdots & \vdots & \vdots \\
\texttt{bop}\left(\texttt{handle}^{\varepsilon(h,E)}\right) & = & \texttt{bop}(E) \cup \{\mathsf{op} \mid (\mathsf{op} \rightarrow f) \in h\}
\end{array}$$

Listing 2: Reduction Rules for `Effects`

CHAPTER 4

Formal Compiler

A compiler from Contracts to `Effects` is straightforward for expressions inherited from the Base language. The compiler keeps these same expressions, albeit each sub-expression must be compiled *recursively* to ensure no contracts remains after compilation. Remaining expressions are contract-related expressions. Our idea is that a contract check is a side effect, defined as $\mathscr{C}$. This effect idealy receives a tuple $\langle l, \langle e, v \rangle \rangle$ of blame label l, contract predicate e, and to-be-checked value v. Then, a flat contract is simly an effect perform with its correct inputs.

Functional contracts are more complex. Because the effect can only be invoked on a flat contract, the compiler recursively expands the expression, pads the expression into a function when it is expecting a function. During the compilation, new variables are generated to pad the expression into a function. The process is also applicable to dependent contracts. However, the dependent contract needs to update the post contract with a contract for argument. This is trivial but requires a duplication of compilation with a different label. The rules are crafted carefully to prevent syntaxs of `Effects` into Contracts while compiling the dependent contracts. The compiled argument contract is delayed substitution, using a random variable as placeholder, until we have the `Effects` expression which we can safely replace into.

Compilation for tuple contracts are straightforward. Each tuple entries are compiled individualy, and reconstructed into a new tuple. Then, each entries are protected, flat contracts are checked immediately during tuple construction; functions are protected and should be checked when used.

4.1 Handler design

After all flat contracts are converted into performing an effect, we need a handler. A simple handler can be installed like below:

$$h_{\text{naive}} = \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } e\ v \text{ then } k\ v \text{ else } \texttt{error}_l\}$$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the value into tuple pattern

This handler performs the check for the value v, with the predicate/contract e. The continuation k is invoked with the value if the check passes, and blame the label l if the contract is violated. At first glance, this handler is correct. In fact, the handler should produce correct semantic, as long as no dependent contracts are used. When dependent contracts are used, e is now populated with monitored arguments, which after compilation should include $\texttt{perform}\ \mathscr{C}\ \tau$ arg. And since the handler is moved into the continuation k, there is no handler at $e\ v$. Take the following program wrapped with the naive handler h_{naive}.

$$\begin{aligned}
&\mathsf{mon}^{k,l}_j(\mathsf{flat}(e_1) \to_i \lambda y.\ \mathsf{flat}(e_2), f)\ 42 \\
&\qquad\qquad \text{where } e_1 = \lambda x.\ \mathsf{true},\ e_2 = \lambda x.x > y, \text{ and } f = \lambda x.x + x \\
\rightarrowtail\quad &(\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle)\ 42 \\
&\qquad\qquad \text{where } e_3 = e_2[y := \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle]
\end{aligned}$$

$$\begin{aligned}
&\texttt{handler}\ h_{\text{naive}}\ \lambda_.(\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle)\ 42) \\
\Longrightarrow\quad &\texttt{handle}\ h_{\text{naive}}\ (\lambda x_1.\ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle\ 42) \\
\Longrightarrow\quad &\texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle)\rangle\rangle \\
&\qquad\qquad \text{where } e_3' = e_2[y := \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle] \\
\Longrightarrow^*\quad &\texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ 42\rangle\rangle \\
\Longrightarrow^*\quad &\texttt{handle}\ h_{\text{naive}}\ \ \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', 84\rangle\rangle \\
\Longrightarrow^*\quad &\text{if } e_3'\ 84 \text{ then } (\lambda x.\ \texttt{handle}\ h_{\text{naive}}\ x)\ 84 \text{ else } \texttt{error}_k \\
\Longrightarrow^*\quad &\text{if } 84 > (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle) \text{ then } (\lambda x.\ \texttt{handle}\ h_{\text{naive}}\ x)\ 84 \text{ else } \texttt{error}_k \\
&\textit{stuck}
\end{aligned}$$

To resolve this issue, we define a recursive function wrap that will reinstall the handler at $e\ v$ making sure all effects during contract checking is handled. Contracts related compilation rules are defined in Figure 11.

4.2 Effects annotation

Careful readers may have noticed that the compiler does not annotate effects. Since the only effects that may occur is the contract checking effect $\mathtt{h}_{\mathscr{C}}$, a lazy compiler can annotate this effect to lambdas, handlers, and performs. By the type system of `Effects`, a function annotated with effect without using the effect can still be typed. Our compiler will produce code annotated with $\langle \mathtt{h}_{\mathscr{C}} \rangle$.

4.3 Compiler Type Preservation and Effect Safety

The compiler should produce same type expression. However, this is problematic when effects are introduced. Recall that the type system separates between pure function, and effectful functions. And type judgement separates between non-effectful computations and effectful computations. Depending on the source, compiled program may have the same type with no effects (no monitors), or same type with some effects (monitors during computation), or purely produce a function wrapped by monitors, or effectfully produce a function wrapped by monitors (monitors during computation).

Theorem D.1. (Compiler Type Preservation)

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then $\cdot \vdash e_2 : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$

Theorem D.3. (Compiler Effect Safety with wrap)

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ and $\cdot \vdash e_2 : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ then $\cdot \vdash \mathsf{wrap}[\tau^*]\ (\lambda_ : \mathsf{unit}.e) : \tau^* \mid \langle\rangle$

Proof. True by type checking. □

4.4 Compiler Correctness

The compiler should produce a program equivalent to the source program. If the source program produces a value, the target program should also produce (the same) value, and with an equivalent storage space.

Theorem E.7. (Compiler Correctness)

If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e^* = \text{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ and $e \twoheadrightarrow^* b$ then $e^* \Longrightarrow^* b$.

$$\frac{\Gamma \vdash e_1 : \tau \to \mathsf{bool} \rightarrowtail e_3 \qquad \Gamma \vdash e_2 : \tau \rightarrowtail e_4}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_3, e_4 \rangle\rangle}\ \text{Compile-Flat}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrowtail e_2}{\Gamma \vdash \lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}\ \text{Compile-Func}$$

$$\frac{\begin{array}{c} f, x \notin \mathrm{dom}(\Gamma) \cup \{y\} \cup \mathrm{fv}(\kappa_1, \kappa_2) \qquad \Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail e_2 \\ e_w = \lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\Big(\Big\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\Big\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\ \Gamma \vdash e_w : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3 \end{array}}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}\ \text{Compile-Dep}$$

$$\frac{\begin{array}{c} x \notin \mathrm{dom}(\Gamma) \qquad \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_1, \mathsf{inj.l}(x)) : \tau_1 \rightarrowtail e_1 \\ \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_2, \mathsf{inj.r}(x)) : \tau_2 \rightarrowtail e_2 \qquad \Gamma \vdash e : \langle \tau_1, \tau_2 \rangle \rightarrowtail e_3 \end{array}}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\langle \kappa_1, \kappa_2 \rangle, e) : \langle \tau_1, \tau_2 \rangle \rightarrowtail (\lambda x.\langle e_1, e_2 \rangle)\ e_3}\ \text{Compile-Tuple}$$

$$\begin{array}{rcl} \mathtt{h}_{\mathscr{C}} : \mathtt{lab} & = & \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha \rangle\rangle \to \alpha\} \\ \mathsf{wrap} & = & \mu w : \forall\alpha.\big(\big(() \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to \alpha\big).\Lambda\alpha.\ \lambda f : \big(() \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big). \\ & & \mathtt{handler}\ \{\mathscr{C} \to \lambda\langle l, \langle e, v \rangle\rangle.\lambda k.\ \text{if } w[\mathsf{bool}]\ (\lambda_.e\ v)\ \text{then } k\ v\ \text{else } \mathtt{error}_l\}\ f \\ & & \text{where } \lambda\langle l, \langle e, v \rangle\rangle \text{ deconstructs the argument as tuple pattern} \end{array}$$

Figure 11: Compiler Rules

4.5 Compiling Contracts for Mutable Cells

4.5.1 Observation

Contracts uses $\mathsf{guard}_j^{k,l}(\kappa, v)$ as a proxy to mutable cells. Get and set operation go through this guard to enfore contracts at run-time. This proxy makes the action oblivious whether it is a real cell or a guarded value. Consider this simple function that calls $\mathsf{get}(c)$ for any cell.

$$\lambda c.\ \mathsf{get}(c)$$

This function should work with both cells $\mathsf{new}(v)$ and guards constructed from $\mathsf{mon}_j^{k,l}(\mathsf{ref/c}(e), r)$ (where r a location). `Effects` does not have special reduction rules for mutable cells, therefore, we need a unified interface between cells and protected cells, and rewrite cell operations to use that unified interface.

4.5.2 Unified Interface

Our compiler has supports for functions and tuples, this is useful for us to design such unified interface. A mutable cell must support get and set, we can encode this as a tuple of 2 functions. Contracts on a cell is now converted into contracts placed on these get and set functions. Cell's location is enclosed inside the closure, and should be accessible when invoked. The compiler should first convert all cell operations into this new wrapper. Then convert all $\mathsf{ref/c}(\kappa)$ into tuple contracts for get and set functions.

$$\frac{\Gamma \vdash e : \tau \rightarrowtail e_1}{\Gamma \vdash \mathsf{new}(e) : \mathsf{ref}(\tau) \rightarrowtail \mathsf{newcell}\ e_1}\ \text{Compile-New}$$

$$\frac{\Gamma \vdash e : \mathsf{ref}(\tau) \rightarrowtail e_1}{\Gamma \vdash \mathsf{get}(e) : \tau \rightarrowtail \mathsf{getcell}\ e_1}\ \text{Compile-Get}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{ref}(\tau) \rightarrowtail e_3 \qquad \Gamma \vdash e_2 : \tau \rightarrowtail e_4}{\Gamma \vdash \mathsf{set}(e_1, e_2) : \tau \rightarrowtail \mathsf{setcell}\ e_3\ e_4}\ \text{Compile-Get}$$

$$\frac{\begin{array}{c} x \notin \Gamma \\ \Gamma, x : \langle \mathsf{unit} \to \tau, \tau \to \mathsf{loc} \rangle \vdash \mathsf{mon}_j^{k,l}(\langle \mathrm{any} \to \kappa, \kappa \to \mathrm{any} \rangle, x) : \langle \mathsf{unit} \to \tau, \tau \to \mathsf{loc} \rangle \rightarrowtail e_1 \\ \Gamma \vdash e : \mathsf{ref}(\tau) \rightarrowtail e_2 \qquad e_3 = e_1[x := e_2] \end{array}}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{ref/c}(\kappa), e) : \mathsf{ref}(\tau) \rightarrowtail e_3}\ \text{Compile-Mon-Ref}$$

$$\begin{array}{lcl} \texttt{newcell} & = & \lambda v.(\lambda l.\langle \lambda_.\ \mathsf{get}(l), \lambda v.\ \mathsf{set}(l, v) \rangle)\ \mathsf{new}(v) \\ \texttt{getcell} & = & \lambda b.\ \mathsf{inj.l}(b)\ \mathsf{unit} \\ \texttt{setcell} & = & \lambda b.\lambda v.(\lambda_.b\ (\mathsf{inj.r}(b)\ v)) \end{array}$$

Listing 3: Compiler Rules for Mutable Cells

CHAPTER 5

Implementation

We implement the formal compiler model into OCaml, by adding a preprocessor. We utilize the OCaml attribute to add contract annotations to variables. The preprocessor rewrites annotated variables according to the compiler rules defined. Note that the compiler does not place a handler during compilation, but after the compilation. Because the handler must be installed around the whole program, more precisely, around everywhere the contract annotated variables are used. The developer can install the handler manually by invoking a predefined wrapper, or annotate a function as main and the preprocessor rewrites to add wrapper automatically.

Annotations are OCaml annotations. Although OCaml annotations support string-like payload, we find it better to use a OCaml type expression as payload to define contracts. Annotations are not code, therefore we cannot support first-order contracts, and use function names as predicates. In Listing 4, we define the contract annotation syntax that developers can use to annotate their OCaml code with contracts.

```OCaml
(* flat contract *)
let [@contract : predicate] x : t = e

(* function contract
 * arguments are inside a tuple
 *)
let [@contract : (p1_pred * p2_pred) -> f_pred]
  f (p1 : t1) (p2 : t2) : t = f_body

(* dependent function contract
 * similarly to function contract, annotated with as 'dep
 *)
let [@contract : (p1_pred * p2_pred) -> g_pred as 'dep]
  g (p1 : t1) (p2 : t2) : t = g_body

(* tuple contract
 * using tuple syntax and annotated with as 'tuple
 *)
let [@contract : (t1_pred * t2_pred) as 'tuple]
  tup : (t1 * t2) = (e, e)
```

Listing 4: OCaml Contract Annotation Example

The preprocessor rewrite using the compiler rules. All flat contracts are compiled into a single effect, formally it is $\mathscr{C}$, implementation wise we define a name `Contract.Check`.

```OCaml
module Contract = struct
type _ Effect.t +=
  | Check : (string * ('a -> bool) * 'a) -> 'a Effect.t
end
```

wrap is defined similarly in OCaml, we remove the type annotations for clarity.

```OCaml
let rec wrap thunk =
  Effect.Deep.match_with thunk ()
  {
    Effect.Deep.retc = (fun x -> x);
    exnc = (fun e -> raise e);
    effc = fun eff ->
      let open Effect.Deep in
      match eff with
```

```
9         | Flat (label, check, arg) ->
10            Some (fun k ->
11              let ok = wrap (fun () -> check arg) in
12              if ok
13              then continue k arg
14              else discontinue k (Blame label))
15        | _ -> None
16    }
```

5.1 Contract States and Interaction

5.1.1 Global State

Because the entire program is wrapped into a single handler for all effects, we can unify a single state for effects only. Predicates must now receive an extra argument, which is the context. The wrap function is now defined with an extra argument current context, and it returns the result and updated context. At the callsite, it is provided with the initial value. States are provided to the handler, therefore the handler provide two effects to get and set the state.

```
1   type state = int                                                    OCaml
2   let state_init = 0
3
4   let rec wrap_inner : type r. (unit -> r) -> state -> r * state =
5     fun thunk init ->
6     match_with thunk ()
7     {
8       retc = (fun x state -> (x, state));
9       exnc = (fun e _state -> raise e);
10      effc = fun (type a) (eff : a Effect.t) ->
11        let handle_eff : ((a, state -> r * state) continuation -> state -> r
          * state) option =
12        match eff with
13        | Get -> Some (fun k state -> continue k state state)
14        | Set x -> Some (fun k _state -> continue k () x)
15        | Flat (label, check, arg) ->
```

```
16            Some (fun k state ->
17              let (ok, updated_state) = wrap_inner (fun () -> check arg)
                state in
18              if ok
19              then continue k arg updated_state
20              else discontinue k (Blame label) updated_state)
21
22        | _ -> None
23      in
24      handle_eff
25    }
26    init
27
28  let wrap main =
29    let r, _ = wrap_inner main state_init in
30    r
31
32  let () = wrap main
```

This approach works, but with a small caveat. Main code can access this state, and can potentially modifies the contract's state. This issue is easily resolved by making two more handlers, one handler for state, and one handler perform recursive wrapping for contract checking code. The state handler is installed in the main wrapper, placing on the top most contract checking code. When a contract check occurs it opens a new scope, starting with the current state. All dependent contract checking code originates from this top most contract update on the same state. Once the top most contract code returns, the state is updated at the continuation of the main wrapper. Because the state is registered by the main wrapper, and the handler for state is registered only when contract checking code executes, main code has no access to this state.

However, in this implementation, a `Flat` effect might be invoked arbitrarily by the main code. In an actual language implementation, we suggest this effect be private, or a special keyword to prevent developers interact with the effect state.

```OCaml
let rec with_state : type res. (unit -> res) -> int -> res * int =
  fun thunk init ->
  match_with thunk ()
  { retc = (fun x s -> (x, s));
    exnc = (fun e _ -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      let handler : ((a, int -> res * int) continuation -> int -> res *
      int) option =
        match eff with
        | Get -> Some (fun k s -> continue k s s)
        | Set x -> Some (fun k _ -> continue k () x)
        | _ -> None
      in handler
  } init

let rec wrap_inner : type res. (unit -> res) -> res =
  fun thunk ->
  match_with thunk ()
  { retc = (fun x -> x);
    exnc = (fun e -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      match eff with
      | Flat (label, check, arg) ->
          Some (fun k ->
            let ok = wrap_inner (fun () -> check arg) in
            if ok then continue k arg
            else discontinue k (Blame label))
      | _ -> None
  }

let rec wrap : type res. (unit -> res) -> int -> res * int =
  fun thunk init ->
  match_with thunk ()
  { retc = (fun x s -> (x, s));
    exnc = (fun e _ -> raise e);
    effc = fun (type a) (eff : a Effect.t) ->
      let handler : ((a, int -> res * int) continuation -> int -> res *
      int) option =
```

```
37          match eff with
38          | Flat (label, check, arg) ->
39              Some (fun k current_state ->
40                let (ok, next_state) =
41                  with_state (fun () -> wrap_inner (fun () -> check arg))
                    current_state
42                in
43                if ok then continue k arg next_state
44                else discontinue k (Blame label) next_state)
45          | _ -> None
46        in handler
47    } init
```

5.2 Contract Interaction

All contract are unified under a single context. This context is accessible through two effects `Get` and `Set`. Let this context be a simple key-value storage, then programmers can devise complicated schemes to track function calls, current function scope, internal contract state, etc, ...

Bibliography

[1] Andrej Bauer and Matija Pretnar. 2014. An Effect System for Algebraic Effects and Handlers. *Log. Methods Comput. Sci.* 10, 4 (2014). https://doi.org/10.2168/LMCS-10(4:9)2014

[2] Christos Dimoulas and Matthias Felleisen. 2011. On contract satisfaction in a higher-order world. *ACM Trans. Program. Lang. Syst.* 33, 5 (2011), 16:1–16:29. https://doi.org/10.1145/2039346.2039348

[3] Christos Dimoulas, Sam Tobin-Hochstadt, and Matthias Felleisen. 2012. Complete Monitors for Behavioral Contracts. In *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science)*, 2012. Springer, 214–233. https://doi.org/10.1007/978-3-642-28869-2_11

[4] Tim Disney, Cormac Flanagan, and Jay McCarthy. 2011. Temporal higher-order contracts. *SIGPLAN Not.* 46, 9 (September 2011), 176–188. https://doi.org/10.1145/2034574.2034800

[5] Robert Bruce Findler and Matthias Felleisen. 2013. ICFP 2002: Contracts for higher-order functions. *ACM SIGPLAN Notices* 48, 4S (2013), 34–45. https://doi.org/10.1145/2502508.2502521

[6] Oleg Kiselyov and KC Sivaramakrishnan. 2018. Eff directly in OCaml. *arXiv preprint arXiv:1812.11664* (2018).

[7] Daan Leijen. 2016. *Algebraic Effects for Functional Programming.* Retrieved from https://www.microsoft.com/en-us/research/publication/algebraic-effects-for-functional-programming/

[8] Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. *SIGPLAN Not.* 52, 1 (January 2017), 486–499. https://doi.org/10.1145/3093333.3009872

[9] Paul Blain Levy. 2013. Call-by-push-value. Doctoral dissertation.

[10] Bertrand Meyer. 1992. The eiffel programming language. *See http://www. eiffel. com* (1992).

[11] Cameron Moy, Christos Dimoulas, and Matthias Felleisen. 2024. Effectful Software Contracts. *Proc. ACM Program. Lang.* 8, POPL (2024), 2639–2666. https://doi.org/10.1145/3632930

[12] Gordon D. Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Appl. Categorical Struct.* 11, 1 (2003), 69–94. https://doi.org/10.1023/A:1023064908962

[13] Matija Pretnar. 2010. Logic and handling of algebraic effects. Doctoral dissertation. Retrieved from https://hdl.handle.net/1842/4611

[14] Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. In *The 31st Conference on the Mathematical Foundations of Programming Semantics, MFPS 2015, Nijmegen, The Netherlands, June 22-25, 2015 (Electronic Notes in Theoretical Computer Science)*, 2015. Elsevier, 19–35. https://doi.org/10.1016/J.ENTCS.2015.12.003

[15] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. *CoRR* abs/2104.00250, (2021). Retrieved from https://arxiv.org/abs/2104.00250

[16] Ningning Xie, Jonathan Immanuel Brachthäuser, Daniel Hillerström, Philipp Schuster, and Daan Leijen. 2020. Effect handlers, evidently. *Proc. ACM Program. Lang.* 4, ICFP (2020), 99:1–99:29. https://doi.org/10.1145/3408981

[17] Dana N Xu. 2014. Dynamic Contract Checking for OCaml.

CHAPTER A

Base Proofs

We prove type safety of Base. The language has no mutable cells, so the store never changes and a step is just $e \longrightarrow e'$.

Each binary operator $o \in \{+, -, \wedge, \vee\}$ has a signature $o : \tau_a \times \tau_b \to \tau_o$ (used by T-Op), with $+, - : \mathsf{num} \times \mathsf{num} \to \mathsf{num}$ and $\wedge, \vee : \mathsf{bool} \times \mathsf{bool} \to \mathsf{bool}$.

A.1 Type Safety

Theorem A.1. (Base Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$
2. If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'.

Proof. By Lemma A.2 and Lemma A.3. □

Lemma A.2. (Base Preservation) If $\cdot \vdash e_1 : \tau$ and $e_1 \longrightarrow e_2$ then $\cdot \vdash e_2 : \tau$.

Proof of Lemma A.2.

$e_1 = E[e_1'], e_2 = E[e_2'], e_1' \dashrightarrow e_2'$	`Step`
$\cdot \vdash e_1' : \tau_1$	Lemma A.5
$\cdot \vdash e_2' : \tau_1$	Lemma A.6
$\cdot \vdash E[e_2'] : \tau$	Lemma A.5

□

Lemma A.3. (Base Progress) If $\cdot \vdash e_1 : \tau$ then e_1 is a value $e_1 \longrightarrow e_2$ for some e_2.

Proof of Lemma A.3. By induction on the derivation of $\cdot \vdash e_1 : \tau$.

- Case T-Var. Impossible, the context is empty.
- Case T-App, $e_1 = e_3\, e_4$. By the I.H. e_3 is a value or steps, and then e_4 is a value or steps. If one of them steps then it steps, else if both are values:

$\cdot \vdash e_3 : \tau_2 \to \tau$	`T-App`
$e_3 = \lambda x : \tau_2.e$	Lemma A.4

$e_3\ e_4 \longrightarrow e[x := e_4]$ S-App

- Other cases are trivial: the expression is a value, or it takes a step by the matching rule of $\dashrightarrow$ (using Lemma A.4 for the shape of values).

□

Lemma A.4. (Canonical forms) If $\cdot \vdash v : \tau$ then:
- If $\tau = \mathsf{num}$ then $v = n$.
- If $\tau = \mathsf{bool}$ then $v = b$.
- If $\tau = \mathsf{unit}$ then $v = \mathsf{unit}$.
- If $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$.
- If $\tau = \langle \tau_1, \tau_2 \rangle$ then $v = \langle v_1, v_2 \rangle$ with $\cdot \vdash v_1 : \tau_1$ and $\cdot \vdash v_2 : \tau_2$.

Proof. By type rules T-Num, T-Bool, T-Unit, T-Lambda, T-Tuple. □

Lemma A.5. (Decomposition) If $\cdot \vdash E[e] : \tau$ then there is τ_1 such that $\cdot \vdash e : \tau_1$, and for every e' with $\cdot \vdash e' : \tau_1$ we have $\cdot \vdash E[e'] : \tau$.

Proof. By induction on E.

- Case $E = \square$. Take $\tau_1 = \tau$.
- Case $E = E_0\ e_0$.

  $\cdot \vdash E_0[e]\ e_0 : \tau$ Given
  $\cdot \vdash E_0[e] : \tau_2 \to \tau$ T-App
  $\cdot \vdash e_0 : \tau_2$ T-App
  $\cdot \vdash E_0[e'] : \tau_2 \to \tau$ I.H.
  $\cdot \vdash E_0[e']\ e_0 : \tau$ T-App
- Other cases are trivial: invert the typing rule of the frame, apply the I.H. to E_0, and apply the same rule again.

□

Lemma A.6. (Redex preservation) If $\cdot \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\cdot \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$.

- Case S-App, $(\lambda x : \tau_1.e)\ v \dashrightarrow e[x := v]$.

  $\cdot \vdash \lambda x : \tau_1.e : \tau_1 \to \tau$ T-App
  $\cdot \vdash v : \tau_1$ T-App
  $x : \tau_1 \vdash e : \tau$ T-Lambda
  $\cdot \vdash e[x := v] : \tau$ Lemma A.7
- Case S-Fix, $\mu x : \tau.e \dashrightarrow e[x := \mu x : \tau.e]$.

$x : \tau \vdash e : \tau$ — T-Fix
$\cdot \vdash \mu x : \tau . e : \tau$ — Given
$\cdot \vdash e[x := \mu x : \tau . e] : \tau$ — Lemma A.7

- Other cases are trivial.

□

Lemma A.7. (Substitution) If $\Gamma, x : \tau_1 \vdash e : \tau$ and $\Gamma \vdash e_1 : \tau_1$ then $\Gamma \vdash e[x := e_1] : \tau$.

Proof. By induction on the derivation of $\Gamma, x : \tau_1 \vdash e : \tau$.

- Case T-Var with $e = x$. Then $\Gamma, x : \tau_1 \vdash x : \tau_1$ and $\tau = \tau_1$.

  $\Gamma \vdash e_1 : \tau_1$ — Given
  $x[x := e_1] = e_1$ — Substitution
  $\Gamma \vdash x[x := e_1] : \tau$

- Case T-Var with $e = y \neq x$. Then $\Gamma, x : \tau_1 \vdash y : \tau$

  $y : \tau \in (\Gamma, x : \tau_1)$ — T-Var
  $y : \tau \in \Gamma$ — $y \neq x$
  $\Gamma \vdash y[x := e_1] : \tau$ — T-Var, $y[x := e_1] = y$

- Case T-Lambda with $e = \lambda y : \tau_2 . e_2$, $y \neq x$ and $y \notin \text{fv}(e_1)$.

  $\Gamma, x : \tau_1, y : \tau_2 \vdash e_2 : \tau_3$ — T-Lambda
  $\Gamma, y : \tau_2, x : \tau_1 \vdash e_2 : \tau_3$ — Unordered Context
  $\Gamma, y : \tau_2 \vdash e_2[x := e_1] : \tau_3$ — I.H.
  $\Gamma \vdash \lambda y : \tau_2 . e_2[x := e_1] : \tau_2 \to \tau_3$ — T-Lambda

- Case T-Fix with $e = \mu y : \tau . e_2$. Same as T-Lambda.

- Case T-App with $e = e_2\ e_3$.

  $\Gamma, x : \tau_1 \vdash e_2 : \tau_2 \to \tau$ — T-App
  $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ — T-App
  $\Gamma \vdash e_2[x := e_1] : \tau_2 \to \tau$ — I.H.
  $\Gamma \vdash e_3[x := e_1] : \tau_2$ — I.H.
  $\Gamma \vdash e_2\ e_3[x := e_1] : \tau$ — T-App

- Other cases are trivial: the expression has no variables, or the I.H. applies to every premise and the same rule is used again.

□

CHAPTER B

Contracts Theorems

Theorem B.1. (Contracts Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \twoheadrightarrow e'$ then $\cdot \vdash e' : \tau$.
2. If $\cdot \vdash e : \tau$ then e is a value or $e = \mathsf{blame}_j^k$ or $e \twoheadrightarrow e'$ for some e'.

Proof. By Lemma B.2 and Lemma B.3. □

Lemma B.2. (Contracts Preservation) If $\cdot \vdash e_1 : \tau$ and $e_1 \twoheadrightarrow e_2$ then $\cdot \vdash e_2 : \tau$.

Proof of Lemma B.2. By cases on the step.

- Case Step-C, $E[e_1'] \twoheadrightarrow E[e_2']$ with $e_1' \dashrightarrow e_2'$.

  $\cdot \vdash e_1' : \tau_1$ — Lemma B.7
  $\cdot \vdash e_2' : \tau_1$ — Lemma B.8
  $\cdot \vdash E[e_2'] : \tau$ — Lemma B.7

- Case Step-C-Blame, $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$.

  $\cdot \vdash \mathsf{blame}_j^k : \tau$ — `Type-C-Blame`

□

Lemma B.3. (Contracts Progress) If $\cdot \vdash e_1 : \tau$ then e_1 is a value, or $e_1 = \mathsf{blame}_j^k$, or there is e_2 with $e_1 \twoheadrightarrow e_2$.

Proof of Lemma B.3. By induction on the derivation of $\cdot \vdash e_1 : \tau$. When an evaluated sub-expression steps, the whole expression steps by Lemma B.4. When an evaluated sub-expression is blame_j^k, the whole expression is $E\left[\mathsf{blame}_j^k\right]$ with $E \neq \square$ and steps by Step-C-Blame. So in each case we only consider the sub-expressions in evaluation position being values.

- Cases of Base. As in Lemma A.3, with one change in T-App: by Lemma B.5 the function may also be a guard.
  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)$. Guard-Func applies.
  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)$. Guard-Dep applies.
- Case Type-C-Blame. $e_1 = \mathsf{blame}_j^k$.

- Case Type-C-Guard. e_1 is a value.
- Case Type-C-Check, $e_1 = \mathsf{check}_j^k(e, v)$. If e is a value, $\cdot \vdash e : \mathsf{bool}$ so $e \in \{\mathsf{true}, \mathsf{false}\}$ by Lemma B.5, and Check-True or Check-False applies.
- Case Type-C-Mon, $e_1 = \mathsf{mon}_j^{k,l}(\kappa, v)$ with $\cdot \vdash \kappa : \mathsf{con}(\tau)$. By cases on the last rule of the contract typing.
  - Type-C-Flat. Mon-Flat applies.
  - Type-C-Fun or Type-C-DepFun. Mon-Guard applies.
  - Type-C-Tuple. $\tau = \langle \tau_1, \tau_2 \rangle$, so $v = \langle v_1, v_2 \rangle$ by Lemma B.5, and Mon-Tuple applies.

□

Lemma B.4. (Steps under contexts) If $e \to e'$ then $E[e] \to e''$ for some e''.

Proof. If the step is Step-C, then $E[e] \to E[e']$ by Step-C with the composed context. If it is Step-C-Blame, $e = E'\left[\mathsf{blame}_j^k\right]$ with $E' \neq \square$, so $E[E'] \neq \square$ and $E[e] \to \mathsf{blame}_j^k$ by Step-C-Blame. □

Lemma B.5. (Canonical forms) Let $\cdot \vdash v : \tau$. The cases of Lemma A.4 hold, except for function types: if $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$ or $v = \mathsf{guard}_j^{k,l}(\kappa, v')$ where κ is $\kappa_1 \to \kappa_2$ or $\kappa_1 \to_i \lambda x.\kappa_2$.

Proof. The only new value is the guard. By Type-C-Guard a guard has the type of its contract, and the side condition makes that a function type. So a guard is never a number, a boolean, unit or a tuple. □

Lemma B.6. (Substitution) Let $\Gamma \vdash e_1 : \tau_1$.
1. If $\Gamma, x : \tau_1 \vdash e : \tau$ then $\Gamma \vdash e[x := e_1] : \tau$.
2. If $\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ then $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$.

Proof. By mutual induction on the two derivations. The cases of Base are those of Lemma A.7.

- Case Type-C-Flat.

  $\Gamma, x : \tau_1 \vdash e : \tau \to \mathsf{bool}$ — `Type-C-Flat`
  $\Gamma \vdash e[x := e_1] : \tau \to \mathsf{bool}$ — I.H. 1
  $\Gamma \vdash \mathsf{flat}(e)[x := e_1] : \mathsf{con}(\tau)$ — `Type-C-Flat`

- Case Type-C-Fun and Type-C-Tuple. The I.H. 2 on both contracts, then the same rule.
- Case Type-C-DepFun with $\kappa = \kappa_1 \to_i \lambda y.\kappa_2$, $y \neq x$ and $y \notin \mathrm{fv}(e_1)$.

$\Gamma, x : \tau_1 \vdash \kappa_1 : \mathsf{con}(\tau_2)$ — Type-C-DepFun
$\Gamma, x : \tau_1, y : \tau_2 \vdash \kappa_2 : \mathsf{con}(\tau_3)$ — Type-C-DepFun
$\Gamma \vdash \kappa_1[x := e_1] : \mathsf{con}(\tau_2)$ — I.H. 2
$\Gamma, y : \tau_2 \vdash \kappa_2[x := e_1] : \mathsf{con}(\tau_3)$ — I.H. 2, weakening of e_1
$\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau_2 \to \tau_3)$ — Type-C-DepFun

- Case Type-C-Mon.

$\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ — Type-C-Mon
$\Gamma, x : \tau_1 \vdash e : \tau$ — Type-C-Mon
$\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$ — I.H. 2
$\Gamma \vdash e[x := e_1] : \tau$ — I.H. 1
$\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e)[x := e_1] : \tau$ — Type-C-Mon

- Cases Type-C-Check and Type-C-Guard. The I.H. on each premise, then the same rule. The side condition of Type-C-Guard is about the shape of κ, which substitution does not change.
- Case Type-C-Blame. blame_j^k has no variables.

□

Lemma B.7. (Decomposition) If $\cdot \vdash E[e] : \tau$ then there is τ_1 such that $\cdot \vdash e : \tau_1$, and for every e' with $\cdot \vdash e' : \tau_1$ we have $\cdot \vdash E[e'] : \tau$.

Proof. By induction on E. The frames of Base are as in Lemma A.5. Two frames are new.

- Case $E = \mathsf{mon}_j^{k,l}(\kappa, E_0)$.

$\cdot \vdash \mathsf{mon}_j^{k,l}(\kappa, E_0[e]) : \tau$ — Given
$\cdot \vdash \kappa : \mathsf{con}(\tau)$ — Type-C-Mon
$\cdot \vdash E_0[e] : \tau$ — Type-C-Mon
$\cdot \vdash e : \tau_1$ — I.H.
$\cdot \vdash E_0[e'] : \tau$ — I.H., for $\cdot \vdash e' : \tau_1$
$\cdot \vdash \mathsf{mon}_j^{k,l}(\kappa, E_0[e']) : \tau$ — Type-C-Mon

- Case $E = \mathsf{check}_j^k(E_0, v)$.

$\cdot \vdash \mathsf{check}_j^k(E_0[e], v) : \tau$ — Given
$\cdot \vdash E_0[e] : \mathsf{bool}$ — Type-C-Check
$\cdot \vdash v : \tau$ — Type-C-Check
$\cdot \vdash e : \tau_1$ — I.H.
$\cdot \vdash E_0[e'] : \mathsf{bool}$ — I.H., for $\cdot \vdash e' : \tau_1$
$\cdot \vdash \mathsf{check}_j^k(E_0[e'], v) : \tau$ — Type-C-Check

□

Lemma B.8. (Redex preservation) If $\cdot \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\cdot \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$. The reductions of Base are as in Lemma A.6, using Lemma B.6 for S-App and S-Fix.

- Case Check-True, $\mathsf{check}^k_j(\mathsf{true}, v) \dashrightarrow v$.

  $\cdot \vdash v : \tau$ Type-C-Check

- Case Check-False, $\mathsf{check}^k_j(\mathsf{false}, v) \dashrightarrow \mathsf{blame}^k_j$.

  $\cdot \vdash \mathsf{blame}^k_j : \tau$ Type-C-Blame

- Case Mon-Flat, $\mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}^k_j(e\ v, v)$.

  $\cdot \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)$ Type-C-Mon
  $\cdot \vdash v : \tau$ Type-C-Mon
  $\cdot \vdash e : \tau \to \mathsf{bool}$ Type-C-Flat
  $\cdot \vdash e\ v : \mathsf{bool}$ T-App
  $\cdot \vdash \mathsf{check}^k_j(e\ v, v) : \tau$ Type-C-Check

- Case Mon-Tuple, $\mathsf{mon}^{k,l}_j(\langle \kappa_1, \kappa_2 \rangle, \langle v_1, v_2 \rangle) \dashrightarrow \langle \mathsf{mon}^{k,l}_j(\kappa_1, v_1), \mathsf{mon}^{k,l}_j(\kappa_2, v_2) \rangle$, with $\tau = \langle \tau_1, \tau_2 \rangle$.

  $\cdot \vdash \langle \kappa_1, \kappa_2 \rangle : \mathsf{con}(\langle \tau_1, \tau_2 \rangle)$ Type-C-Mon
  $\cdot \vdash \langle v_1, v_2 \rangle : \langle \tau_1, \tau_2 \rangle$ Type-C-Mon
  $\cdot \vdash \kappa_1 : \mathsf{con}(\tau_1), \cdot \vdash \kappa_2 : \mathsf{con}(\tau_2)$ Type-C-Tuple
  $\cdot \vdash v_1 : \tau_1, \cdot \vdash v_2 : \tau_2$ T-Tuple
  $\cdot \vdash \mathsf{mon}^{k,l}_j(\kappa_i, v_i) : \tau_i$ Type-C-Mon
  $\cdot \vdash \langle \mathsf{mon}^{k,l}_j(\kappa_1, v_1), \mathsf{mon}^{k,l}_j(\kappa_2, v_2) \rangle : \langle \tau_1, \tau_2 \rangle$ T-Tuple

- Case Mon-Guard, $\mathsf{mon}^{k,l}_j(\kappa, v) \dashrightarrow \mathsf{guard}^{k,l}_j(\kappa, v)$ with κ a function contract.

  $\cdot \vdash \kappa : \mathsf{con}(\tau)$ Type-C-Mon
  $\cdot \vdash v : \tau$ Type-C-Mon
  $\cdot \vdash \mathsf{guard}^{k,l}_j(\kappa, v) : \tau$ Type-C-Guard

- Case Guard-Func, $\mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}^{k,l}_j\big(\kappa_2, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\big)$.

  $\cdot \vdash \mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v_1) : \tau_1 \to \tau$ T-App
  $\cdot \vdash v_2 : \tau_1$ T-App
  $\cdot \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau)$ Type-C-Guard
  $\cdot \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard
  $\cdot \vdash \kappa_1 : \mathsf{con}(\tau_1), \cdot \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-Fun

$\cdot \vdash \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau_1$ Type-C-Mon
$\cdot \vdash v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau$ T-App
$\cdot \vdash \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

- Case Guard-Dep, $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$ with $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$.

  $\cdot \vdash v_2 : \tau_1$ T-App
  $\cdot \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard
  $\cdot \vdash \kappa_1 : \mathsf{con}(\tau_1)$ Type-C-DepFun
  $x : \tau_1 \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-DepFun
  $\cdot \vdash \mathsf{mon}_j^{l,j}(\kappa_1, v_2) : \tau_1$ Type-C-Mon
  $\cdot \vdash \kappa_3 : \mathsf{con}(\tau)$ Lemma B.6 (2)
  $\cdot \vdash \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau_1$ Type-C-Mon
  $\cdot \vdash v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau$ T-App
  $\cdot \vdash \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

  The dependent variable x is replaced by a monitored expression, not a value. This is fine because Contracts has no effects in its types, and Lemma B.6 allows any expression of the right type.

□

CHAPTER C

Contract Erasure Theorem

Define erasure compiler E as a homomorphic function over expressions of Contracts with a single transformation:

$\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau \rightarrowtail e$

C.1 Relation

Define Contracts F as contexts without $\mathsf{check}_j^k(E, v)$.

Define these relations:

- $R_e \subseteq$ Contracts $e \times$ Base e

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $F \times$ Base E

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

- $R_v \subseteq$ Contracts $v \times$ Base v

  If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We define $\tilde{v}$ as shorthand for any variable $\tilde{v}$ satisfying $v \sim \tilde{v}$.

$\boxed{\sim \subseteq \text{Contracts } F \times \text{Base } E}$

Define $\tilde{F}$ such that $F \sim \tilde{F}$.

$$\begin{aligned} \square &\sim \square \\ F\ e &\sim \tilde{F}\ \tilde{e} \\ v\ F &\sim \tilde{v}\ \tilde{F} \\ \mathsf{mon}_j^{k,l}(\kappa, F) &\sim \tilde{F} \end{aligned}$$

$\sim \subseteq$ Contracts $e \times$ Base e

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\kappa, e_2) & \sim & \tilde{e}_2 \\
\mathsf{check}_j^{k}(e_1, v) & \sim & \tilde{v}
\end{array}
$$

$\sim \subseteq$ Contracts $v \times$ Base v

$$
\begin{array}{rcl}
n & \sim & n \\
b & \sim & b \\
x & \sim & x \\
\lambda x.e & \sim & \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa, v) & \sim & \tilde{v}
\end{array}
$$

Lemma C.1. (Relation is closed under check-free contexts)

If $e \sim \tilde{e}$ and $F \sim \tilde{F}$, then $F[e] \sim \tilde{F}[\tilde{e}]$.

Proof.

By induction on $F \sim \tilde{F}$.

- Case $\square \sim \square$

$e \sim \tilde{e}$	Given
$F[e] = e$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{e}$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	

- Case $F_1\ e_2 \sim \tilde{F}_1\ e_2$

$e \sim \tilde{e}$	Given
$F_1[e] \sim \tilde{F}_1[\tilde{e}]$	I.H.
$F[e] = F_1[e]\ e_2$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{F}_1[\tilde{e}]\ \tilde{e}_2$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	definition

- Similarly for other cases

□

Lemma C.2. (Relation is closed under substitution)

If $e \sim \tilde{e}$ and $v \sim \tilde{v}$, then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$ for every x.

Proof.

By I.H. on $e \sim \tilde{e}$.

- Case $x \sim x$

$v \sim \tilde{v}$	Given
$e[x := v] = v$	Replacement
$\tilde{e}[x := \tilde{v}] = \tilde{v}$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	By $v \sim \tilde{v}$

- Case $y \sim y$, where $y \neq x$

$v \sim \tilde{v}$	Given
$e[x := v] = y$	Replacement
$\tilde{e}[x := \tilde{v}] = y$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	By $y \sim y$

- Case $e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$

$e_1[x := v] \sim \tilde{e}_1[x := \tilde{v}]$	I.H.
$e_2[x := v] \sim \tilde{e}_2[x := \tilde{v}]$	I.H.
$e[x := v] = e_1[x := v]\ e_2[x := v]$	Replacement
$\tilde{e}[x := v] = \tilde{e}_1[x := \tilde{v}]\ \tilde{e}_2[x := \tilde{v}]$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	Definition

- Similarly with other cases. Lambda case avoid same variable by replacement $e[x := v]$ to free variables of e.

□

Lemma C.3. (Evaluation-context decomposition)

Forall context E, either:

- $E = F$ or
- $E = F\left[\mathsf{check}_j^k(E', v)\right]$

Proof. By induction on E.

- Case $\square$

$F = \square$	Choose
$E = F$	

- Case $E = \mathsf{check}_j^k(E_1, v)$

$F = \square$ Choose
$E' = E_1$ Choose
$E = F\left[\mathsf{check}_j^k(E_1, v)\right]$

- Case $E_1\ e$

  I.H. on E_1:

  ▸ Case $E_1 = F_1$

  $E = F_1\ e$
  $F = F_1\ e$ Choose
  $E = F$

  ▸ Case $E_1 = F_1\left[\mathsf{check}_j^k(E_1', v)\right]$

  $E = F_1\left[\mathsf{check}_j^k(E_1', v)\right]\ v$
  $F = F_1\ v$ Choose
  $E' = E_1'$ Choose
  $E = F\left[\mathsf{check}_j^k(E_1', v)\right]$

- Other cases follows similarly

□

$$\mathrm{ER}(e, e') = \begin{cases} \text{true if } e{\sim}e' \text{ and } e{\neq}F\left[\mathsf{check}_j^k(e_0, v)\right] \\ \text{true if } e{=}F\left[\mathsf{check}_j^k(e_0, v)\right] \text{ and } e'{=}\tilde{F}[\tilde{v}] \\ \text{false otherwise} \end{cases}$$

Lemma C.4. (Erasure satisfies the state predicate)

If $e' = \mathrm{erase}(e)$, then $\mathrm{ER}(e, e')$.

Proof. I.H on $e' = \mathrm{erase}(e)$.

Because the compiler doesn't compile check (runtime expression), we never get the second clause of EQ. Therefore, $e \neq F\left[\mathsf{check}_j^k(e_0, v)\right]$, and all child nodes of e also not this form.

- Case $e' = \mathrm{erase}\left(\mathsf{mon}_j^{k,l}(\kappa, e_1)\right) = \mathrm{erase}(e_1) = e_1'$

  $\mathrm{ER}(e_1, e_1')$ I.H.
  $e_1 \sim e_1'$ First clause EQ
  $\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim \tilde{e}_1$ Definition
  $e_1 \sim \tilde{e}_1$ Definition
  $\tilde{e}_1 = e_1'$ Choose
  $\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim e_1'$
  $\mathrm{ER}(e, e')$ First clause EQ

- Case $e' = \text{erase}(e_1\ e_2) = \text{erase}(e_1)\ \text{erase}(e_2) = e_1'\ e_2'$

$\text{ER}(e_1, e_1')$	I.H.
$\text{ER}(e_2, e_2')$	I.H.
$e_1 \sim e_1'$	First clause EQ
$e_2 \sim e_2'$	First clause EQ
$e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$	Definition
$\text{ER}(e, e')$	First clause EQ

- Other cases follow similarly to the above case.

□

C.2 Main Theorem Proof

Figure 12 shows the structure of the proof. Erasure initializes the state predicate $\text{ER}(e, e')$ by Lemma C.4. Each non-blaming source step $e_i \twoheadrightarrow e_{i+1}$ is matched by zero or more target steps $e_i' \longrightarrow^* e_{i+1}'$ while preserving $\text{ER}(e_{i+1}, e_{i+1}')$ (Lemma C.7). Chaining these simulations with Lemma C.6 shows that whenever the source reaches b, the erased target reaches the same b.

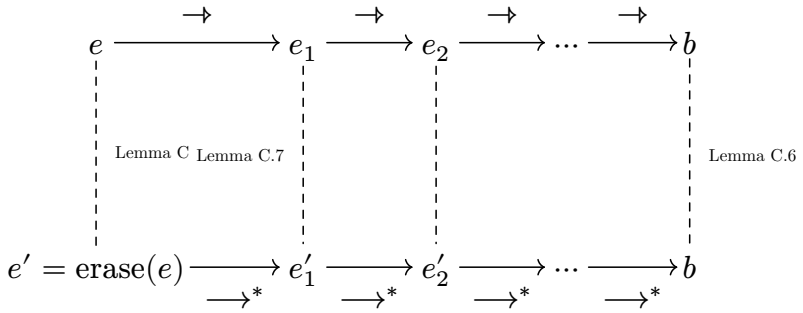


Figure 12: Structure of the contract erasure proof. Dashed vertical lines denote ER.

Theorem C.5. (Contract Erasure Theorem)

If $e' = \text{erase}(e)$ and $e \twoheadrightarrow^* b$, then $e' \longrightarrow^* b$.

Proof.

$\text{ER}(e, e')$	Lemma C.4
$e' \longrightarrow^* b$	Lemma C.6

□

Lemma C.6. (Erasure Multistep Simulation)

If $\mathrm{ER}(e, e')$ and $e \twoheadrightarrow^* b$ then $e' \longrightarrow^* b$.

Proof.

By induction on $e \twoheadrightarrow^* b$.

- Case $b \twoheadrightarrow^* b$

$\mathrm{ER}(e, e')$	Given
$e \sim e'$	First clause EQ
$e' = b$	Relation inversion $b \sim e'$
$e' \longrightarrow^* b$	Reflexivity

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* b$

  By cases on $e \twoheadrightarrow e_1$.

  ▸ Case $e = E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k = e_1$ for some $E \neq \square, k, j$

$e_1 \twoheadrightarrow^* \mathsf{blame}_j^k$	Reflexivity
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

  ▸ Case $e = E[e_2] \twoheadrightarrow E[e_3] = e_1$ where $e_2 \dashrightarrow e_3$ is a redex, and $e_2 \neq \mathsf{blame}_j^k$.

$\mathrm{ER}(e, e')$	Given
$e \twoheadrightarrow e_1$	Given
$e \neq E\left[\mathsf{blame}_j^k\right]$	Case assumption

  – Case $e_2 = \mathsf{check}_j^k(\mathsf{false}, v)$, and $e_3 = \mathsf{blame}_j^k$

$e_1 \twoheadrightarrow \mathsf{blame}_j^k$	Step-C
$e_1 \twoheadrightarrow^* \mathsf{blame}_j^k$	
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

  – Other cases, $e_2 \neq \mathsf{check}_j^k(\mathsf{false}, v)$, and $e_3 \neq \mathsf{blame}_j^k$

  Relation is not local so we run simulation on $e \twoheadrightarrow e_1$.

$e \neq E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]$	Generalize
$e' \longrightarrow^* e_1'$	Lemma C.7
$\mathrm{ER}(e_1, e_1')$	Lemma C.7
$e_1' \longrightarrow^* b$	I.H.
$e' \longrightarrow^* b$	Transitive

□

Lemma C.7. (Erasure Step Simulation)

If $\mathrm{ER}(e_1, e_1')$, $e_1 \twoheadrightarrow e_2$, and $e_1 \neq E_{1a}\left[\mathsf{blame}_{j_{1a}}^{k_{1a}}\right]$ and $e_1 \neq E_{1b}\left[\mathsf{check}_{j_{1b}}^{k_{1b}}(\mathsf{false}, v)\right]$, then there is e_2' such that $e_1' \longrightarrow^* e_2'$ and $\mathrm{ER}(e_2, e_2')$.

Proof.

By case analysis of $\mathrm{ER}(e_1, e_1')$:

- Case $e_1 \sim e_1'$ and $e_1 \neq F\left[\mathsf{check}_{j_1}^{k_1}(e_{1a}, v)\right]$
- Case $e_1 = F\left[\mathsf{check}_{j_1}^{k_1}(e_{1a}, v)\right]$ and $e_1' = \tilde{F}[\tilde{v}]$
  - Case $e_{1a} = \mathsf{true}$

    $e_1 \twoheadrightarrow F[v]$
  - Case $e_{1a} = \mathsf{false}$
  - Case $e_{1a} = v$, and $v \neq b$. Impossible, no rule.
  - Case $e_{1a} \dashrightarrow e_{1b}$

By case analysis of $e_1 \twoheadrightarrow e_2$ with $e_1 \neq E_1\left[\mathsf{blame}_{j_1}^{k_1}\right]$.

$e_1 = E[e_3]$	`C-Step`
$e_2 = E[e_4]$	`C-Step`
$e_3 \dashrightarrow e_4$	`C-Step`
$e_1' = \tilde{E}[\tilde{e}_3]$	Lemma C.8

By cases on the redex reduction $e_3 \dashrightarrow e_4$.

- Case $(\lambda x.e_5)\, v_5 \dashrightarrow e_5[x := v_5]$

$\tilde{e}_3 = (\lambda x.\tilde{e}_5)\, \tilde{v}_5$	definition of $\sim$
$(\lambda x.\tilde{e}_5)\, \tilde{v}_5 \dashrightarrow \tilde{e}_5[x := \tilde{v}_5]$	`S-App`
$e_4 \sim \tilde{e}_5[x := \tilde{v}_5]$	Lemma C.2
$e_1' \longrightarrow^* \tilde{E}[\tilde{e}_4] = e_2'$	reflexivity
$e_2 \sim e_2'$	Lemma C.1

- Case $\mu x : \tau.e_3 \dashrightarrow e_3[x := \mu x : \tau.e_3]$

  Similar to the preceding case.

- Case $\mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v$

$\tilde{e}_3 = \tilde{v}$	definition of $\sim$
$e_4 \sim \tilde{v}$	definition of $\sim$
$e_1' \longrightarrow^* \tilde{E}[\tilde{v}] = e_2'$	reflexivity
$e_2 \sim e_2'$	Lemma C.1

- Case $\mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k$

  $e_2 = E\left[\mathsf{blame}_j^k\right]$ — C-Step
  $e_2 \neq E_2\left[\mathsf{blame}_{j_2}^{k_2}\right]$ — Given
  $\bot$ — impossible

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), v_3) \dashrightarrow \mathsf{check}_j^k(e_3\ v_3, v_3)$

  $\tilde{e}_3 = \tilde{v}_3$ — definition of $\sim$
  $e_4 \sim \tilde{v}_3$ — definition of $\sim$
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_3] = e_2'$ — reflexivity
  $e_2 \sim e_2'$ — Lemma C.1

- Case $\mathsf{mon}_j^{k,l}(\kappa, v_3) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa, v_3)$

  $\tilde{e}_3 = \tilde{v}_3$ — definition of $\sim$
  $e_4 \sim \tilde{v}_3$ — definition of $\sim$
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_3] = e_2'$ — reflexivity
  $e_2 \sim e_2'$ — Lemma C.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

  $\tilde{e}_3 = \tilde{v}_1\ \tilde{v}_2$ — definition of $\sim$
  $\tilde{e}_4 \sim \tilde{v}_1\ \tilde{v}_2$ — definition of $\sim$
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_1\ \tilde{v}_2] = e_2'$ — reflexivity
  $e_2 \sim e_2'$ — Lemma C.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$, where $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

  Similar to the preceding case.

□

Lemma C.8. (Relation Decomposition Through Contexts)

If $e_1 \sim e_1'$, $e_1 = E[e_2]$, and e_2 is a redex, then there are $\tilde{E}$ and $\tilde{e}_2$ such that $E \sim \tilde{E}$ and $e_2 \sim \tilde{e}_2$ and $e_1' = \tilde{E}[\tilde{e}_2]$.

Proof. □

Lemma C.9. (Successful Erasure Simulation)

If $e_1 \sim e_1'$ and $e_1 \to e_2$, where e_2 is not a blaming expression, then there exists an e_2' such that $e_1' \stackrel{*}{\Longrightarrow} e_2'$ and $e_2 \sim e_2'$.

The target may take zero, one, or more steps. In particular, successful check reductions are stuttering steps: they may be simulated by zero target steps.

Lemma C.10. (Successful Evaluation Is Preserved by Erasure)

If $e \sim e'$ and $e \xrightarrow{*} v$ without reaching blame_j^k, then there exists a target value v' such that $e' \xRightarrow{*} v'$ and $v \sim v'$.

The proofs for Effects follow the proofs of System F^{ε} in Effect Handlers, Evidently [16]. Effects is F^{ε} with the expressions of Base, errors, and fixpoints. Its type rules are those of F^{ε}, with the evidence translation left out. This part has four sections: evaluation contexts, substitution, progress, and preservation.

C.3 Conventions

Typing. There is no type variable context. As in F^{ε}, type variables are kept apart by the side conditions $\alpha \notin \mathrm{ftv}(\Gamma)$ of T-E-Perform and $\alpha \notin \mathrm{ftv}(\varepsilon, \tau)$ of T-E-Ops, and every type is assumed well-formed. The rules of Base are lifted to Effects by giving every premise and the conclusion the same effect row ε. The value forms n, b, unit and x are typed by value rules (T-Num, T-Bool, T-Unit, T-E-Var). The effect row is also recorded in effect-bearing syntax: $\mathsf{lambda}^{\varepsilon} x : \tau.e$ records the latent effect of its body, $\texttt{perform}\ \varepsilon\ \texttt{op}\ \tau_s$ records the tail of its operation row, and $\texttt{handler}\ h^{\varepsilon}$ and $\texttt{handle}\ h\ e^{\varepsilon}$ record the handler residual row. Thus $\texttt{handle}\ h\ e^{\varepsilon}$ checks e at $\langle l \mid \varepsilon \rangle$ and produces effect ε. A tuple of values also has a value rule, T-E-VTuple: from $\Gamma \vdash_{\mathrm{val}} v_1 : \tau_1$ and $\Gamma \vdash_{\mathrm{val}} v_2 : \tau_2$ conclude $\Gamma \vdash_{\mathrm{val}} \langle v_1, v_2 \rangle : \langle \tau_1, \tau_2 \rangle$. Without it, a tuple could not be substituted for a variable in a value position, such as the body of $\Lambda\alpha.v$. In the figure, the rule for λ is named T-E-Abs.

Errors. $\texttt{error}_p$ is a value, typed by T-E-Error: $\Gamma \vdash_{\mathrm{val}} \texttt{error}_p : \tau$ for any τ. Step-E-Error aborts the program, $E[\texttt{error}_p] \Longrightarrow \texttt{error}_p$ for $E \neq \Box$. We drop the context $\texttt{error}_E$ of the figure, since the argument of an error is a party, not an expression.

Effect rows and signatures. Rows are lists of labels and may contain duplicates. For a list of labels $L = l_n, ..., l_1$, the row $\langle L \mid \varepsilon \rangle$ is $\langle l_n \mid ...\langle l_1 \mid \varepsilon \rangle ... \rangle$. Unlike F^{ε}, we do not quotient rows by permutation: the rows below are built by the same ordered concatenation. As in [16], operation names are globally unique, so $\texttt{op} \in \Sigma(l)$ determines l, and T-E-Ops gives a handler for l exactly one clause for every operation in $\Sigma(l)$.

Fixpoints. In Effects, a fixpoint is a recursive value of function or polymorphic type:

T-E-Fix: from $\Gamma, x : \tau \vdash_{\text{val}} v : \tau$ conclude $\Gamma \vdash_{\text{val}} \mu x : \tau.v : \tau$, where τ is $\tau_1 \to^{\varepsilon} \tau_2$ or $\forall \alpha^k.\tau_1$

Step-E-Fix-App: $(\mu x : \tau.v)\ v_0 \rightarrow v[x := \mu x : \tau.v]\ v_0$

Step-E-Fix-TApp: $(\mu x : \tau.v)[\tau_s] \rightarrow v[x := \mu x : \tau.v][\tau_s]$

Remark. The lifted fixpoint of Base, which unfolds $\mu x : \tau.e$ to $e[x := \mu x : \tau.e]$ for any e, does not preserve types once effects are tracked. Let $\tau = \mathsf{unit} \to^{\langle\rangle} \mathsf{num}$ and

$e = \left(\mathsf{lambda}^{\langle l \rangle} y.\mathsf{lambda}^{\langle\rangle}_ : \mathsf{unit}.x\ \mathsf{unit}\right)$ perform $\langle\rangle\ \mathsf{op}\ \tau_s\ v$.

Then $x : \tau \vdash e : \tau \mid \langle l \rangle$, so $\mu x : \tau.e$ has type τ with effect $\langle l \rangle$. After one step, $\mu x : \tau.e$ sits in the body of $\lambda_ : \mathsf{unit}....$, which has effect $\langle\rangle$. But $\mu x : \tau.e$ can only be typed with the effect of the perform in e, so the result is not well typed. Restricting the body to a value avoids this. A value can be given any effect, and so can a recursive value. The wrap handler of the compiler is such a recursive value.

C.4 Evaluation Contexts

$\lceil E \rceil$ collects the labels of the handlers in E, innermost first. bop(E) collects the operations they handle.

$$\lceil F_0 \cdot \text{handle } h_1 \cdot F_1 \cdot ... \cdot \text{handle } h_n \cdot F_n \rceil = l_n, ..., l_1 \quad \text{where } h_i : \Sigma(l_i)$$

$$\text{bop}(\square) = \varnothing \qquad \text{bop}(F[E]) = \text{bop}(E) \qquad \text{bop}(\text{handle } h\ e^{\varepsilon}) = \text{bop}(E) \cup \{\mathsf{op} \mid (\mathsf{op} \to f) \in h\}$$

where F is a single frame without a handler. A handler $h : \Sigma(l)$ has a clause for every operation of l (T-E-Ops). So a context E handles $\mathsf{op} \in \Sigma(l)$, i.e. $\mathsf{op} \in$ bop(E), exactly when l occurs in $\lceil E \rceil$.

The judgment $\Gamma \vdash_{\text{ec}} E : \tau_1 \to \tau_2 \mid \varepsilon$ types a context as a function from the type of its hole to the type of the whole expression, where the whole expression has effect ε.

$$\frac{}{\Gamma \vdash_{\text{ec}} \square : \tau \to \tau \mid \varepsilon}\ \text{CE-Empty}$$

$$\frac{\Gamma \vdash e : \tau_2 \mid \varepsilon \qquad \Gamma \vdash_{\text{ec}} E : \tau_1 \to (\tau_2 \to^{\varepsilon} \tau_3) \mid \varepsilon}{\Gamma \vdash_{\text{ec}} E\, e : \tau_1 \to \tau_3 \mid \varepsilon}\ \text{CE-App1}$$

$$\frac{\Gamma \vdash_{\text{val}} v : \tau_2 \to^{\varepsilon} \tau_3 \qquad \Gamma \vdash_{\text{ec}} E : \tau_1 \to \tau_2 \mid \varepsilon}{\Gamma \vdash_{\text{ec}} v\, E : \tau_1 \to \tau_3 \mid \varepsilon}\ \text{CE-App2}$$

$$\frac{\Gamma \vdash_{\text{ec}} E : \tau_1 \to \forall \alpha.\tau_2 \mid \varepsilon}{\Gamma \vdash_{\text{ec}} E[\tau] : \tau_1 \to \tau_2[\alpha := \tau] \mid \varepsilon}\ \text{CE-TApp}$$

$$\frac{\Gamma \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon \qquad \Gamma \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \langle l \mid \varepsilon \rangle}{\Gamma \vdash_{\text{ec}} \texttt{handle}\ h\ e^{\varepsilon} : \tau_1 \to \tau \mid \varepsilon}\ \text{CE-Handle}$$

$$\frac{\Gamma \vdash_{\text{ec}} E : \tau_1 \to \mathsf{bool} \mid \varepsilon \qquad \Gamma \vdash e_2 : \tau \mid \varepsilon \qquad \Gamma \vdash e_3 : \tau \mid \varepsilon}{\Gamma \vdash_{\text{ec}} \text{if } E \text{ then } e_2 \text{ else } e_3 : \tau_1 \to \tau \mid \varepsilon}\ \text{CE-If}$$

$$\frac{\Gamma \vdash_{\text{ec}} E : \tau_1 \to \tau_a \mid \varepsilon \qquad \Gamma \vdash e : \tau_b \mid \varepsilon}{\Gamma \vdash_{\text{ec}} Eoe : \tau_1 \to \tau_o \mid \varepsilon}\ \text{CE-Op1}$$

$$\frac{\Gamma \vdash v : \tau_a \mid \varepsilon \qquad \Gamma \vdash_{\text{ec}} E : \tau_1 \to \tau_b \mid \varepsilon}{\Gamma \vdash_{\text{ec}} voE : \tau_1 \to \tau_o \mid \varepsilon}\ \text{CE-Op2}$$

Figure 13: Typing of evaluation contexts. The frames $\mathsf{zero?}(E)$, $\langle E, e\rangle$, $\langle v, E\rangle$, $\mathsf{inj.l}(E)$, $\mathsf{inj.r}(E)$ have rules CE-Zero, CE-Tuple1, CE-Tuple2, CE-Inj-Left, CE-Inj-Right of the same form, following the type rule of the frame.

Only CE-Handle changes the effect: the hole of $\texttt{handle}\ h\ e^{\varepsilon}$ is typed under $\langle l \mid \varepsilon\rangle$. Following the definitions, for every context

$$\langle \lceil E\, e \rceil \mid \varepsilon\rangle = \langle \lceil E \rceil \mid \varepsilon\rangle \qquad \langle \lceil \texttt{handle}\ h\ e^{\varepsilon} \rceil \mid \varepsilon\rangle = \langle \lceil E \rceil \mid \langle l \mid \varepsilon\rangle\rangle$$

and likewise for every frame without a handler.

C.5 Statements and proofs

Lemma C.11. (Evaluation context typing) If $\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$ and $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon\rangle$ then $\cdot \vdash E[e] : \tau \mid \varepsilon$.

Proof. By induction on the derivation of $\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$.

- Case CE-Empty. $E[e] = e$ and $\langle \lceil \square \rceil \mid \varepsilon \rangle = \varepsilon$.
- Case CE-App1, $E = E_0\ e_0$.

$\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to (\tau_2 \to^{\varepsilon} \tau) \mid \varepsilon$	CE-App1
$\cdot \vdash e_0 : \tau_2 \mid \varepsilon$	CE-App1
$\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \varepsilon \rangle$	given, $\lceil E \rceil = \lceil E_0 \rceil$
$\cdot \vdash E_0[e] : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon$	I.H.
$\cdot \vdash E_0[e]\ e_0 : \tau \mid \varepsilon$	T-E-App

- Case CE-App2, $E = v\ E_0$.

$\cdot \vdash_{\text{val}} v : \tau_2 \to^{\varepsilon} \tau$	CE-App2
$\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \tau_2 \mid \varepsilon$	CE-App2
$\cdot \vdash E_0[e] : \tau_2 \mid \varepsilon$	I.H.
$\cdot \vdash v : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon$	T-E-Val
$\cdot \vdash v\ E_0[e] : \tau \mid \varepsilon$	T-E-App

- Case CE-TApp, $E = E_0[\tau_0]$ and $\tau = \tau_2[\alpha := \tau_0]$.

$\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \forall \alpha.\tau_2 \mid \varepsilon$	CE-TApp
$\cdot \vdash E_0[e] : \forall \alpha.\tau_2 \mid \varepsilon$	I.H.
$\cdot \vdash E_0[e][\tau_0] : \tau_2[\alpha := \tau_0] \mid \varepsilon$	T-E-Tapp

- Case CE-Handle, $E = \texttt{handle}\ h\ e^{\varepsilon}$.

$\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$	CE-Handle
$\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \tau \mid \langle l \mid \varepsilon \rangle$	CE-Handle
$\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \langle l \mid \varepsilon \rangle \rangle$	given, def. of $\lceil \cdot \rceil$
$\cdot \vdash E_0[e] : \tau \mid \langle l \mid \varepsilon \rangle$	I.H.
$\cdot \vdash \texttt{handle}\ h\ e^{\varepsilon} : \tau \mid \varepsilon$	T-E-Handle

- Case CE-If, $E = \text{if } E_0 \text{ then } e_2 \text{ else } e_3$.

$\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \mathsf{bool} \mid \varepsilon$	CE-If
$\cdot \vdash e_2 : \tau \mid \varepsilon, \cdot \vdash e_3 : \tau \mid \varepsilon$	CE-If
$\cdot \vdash E_0[e] : \mathsf{bool} \mid \varepsilon$	I.H.
$\cdot \vdash \text{if } E_0[e] \text{ then } e_2 \text{ else } e_3 : \tau \mid \varepsilon$	T-If

- Cases CE-Op1, CE-Op2, CE-Zero, CE-Tuple1, CE-Tuple2, CE-Inj-Left, CE-Inj-Right. As CE-If: the I.H. on E_0, then the lifted rule of Base for the frame.

□

Lemma C.12. (Effect corresponds to the evaluation context) If $\cdot \vdash E[e] : \tau \mid \varepsilon$ then there is τ_1 such that $\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$ and $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$.

Proof. By induction on E.

- Case $E = \square$. Take $\tau_1 = \tau$, and use CE-Empty.
- Case $E = E_0\ e_0$.

  $\cdot \vdash E_0[e] : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon$ — T-E-App
  $\cdot \vdash e_0 : \tau_2 \mid \varepsilon$ — T-E-App
  $\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to (\tau_2 \to^{\varepsilon} \tau) \mid \varepsilon$ — I.H.
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \varepsilon \rangle$ — I.H.
  $\cdot \vdash_{\text{ec}} E_0\ e_0 : \tau_1 \to \tau \mid \varepsilon$ — CE-App1
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$ — $\lceil E \rceil = \lceil E_0 \rceil$
- Case $E = v\ E_0$.

  $\cdot \vdash v : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon$ — T-E-App
  $\cdot \vdash_{\text{val}} v : \tau_2 \to^{\varepsilon} \tau$ — inversion of T-E-Val
  $\cdot \vdash E_0[e] : \tau_2 \mid \varepsilon$ — T-E-App
  $\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \tau_2 \mid \varepsilon$ — I.H.
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \varepsilon \rangle$ — I.H.
  $\cdot \vdash_{\text{ec}} v\ E_0 : \tau_1 \to \tau \mid \varepsilon$ — CE-App2

  The value v has a function type, so it is not a tuple, and its derivation ends in T-E-Val.
- Case $E = E_0[\tau_0]$.

  $\cdot \vdash E_0[e] : \forall \alpha.\tau_2 \mid \varepsilon$ — T-E-Tapp, $\tau = \tau_2[\alpha := \tau_0]$
  $\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \forall \alpha.\tau_2 \mid \varepsilon$ — I.H.
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \varepsilon \rangle$ — I.H.
  $\cdot \vdash_{\text{ec}} E_0[\tau_0] : \tau_1 \to \tau \mid \varepsilon$ — CE-TApp
- Case $E = \texttt{handle}\ h\ e^{\varepsilon}$ with $h : \Sigma(l)$.

  $\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$ — T-E-Handle
  $\cdot \vdash E_0[e] : \tau \mid \langle l \mid \varepsilon \rangle$ — T-E-Handle
  $\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \tau \mid \langle l \mid \varepsilon \rangle$ — I.H.
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \langle l \mid \varepsilon \rangle \rangle$ — I.H.
  $\cdot \vdash_{\text{ec}} \texttt{handle}\ h\ e^{\varepsilon} : \tau_1 \to \tau \mid \varepsilon$ — CE-Handle
  $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$ — def. of $\lceil \cdot \rceil$
- Case $E = \text{if } E_0 \text{ then } e_2 \text{ else } e_3$.

  $\cdot \vdash E_0[e] : \mathsf{bool} \mid \varepsilon$ — T-If
  $\cdot \vdash e_2 : \tau \mid \varepsilon, \cdot \vdash e_3 : \tau \mid \varepsilon$ — T-If
  $\cdot \vdash_{\text{ec}} E_0 : \tau_1 \to \mathsf{bool} \mid \varepsilon$ — I.H.

$\cdot \vdash e : \tau_1 \mid \langle \lceil E_0 \rceil \mid \varepsilon \rangle$ I.H.

$\cdot \vdash_{\text{ec}} \text{if } E_0 \text{ then } e_2 \text{ else } e_3 : \tau_1 \to \tau \mid \varepsilon$ CE-If

- Cases of the other frames of Base. As if E_0 then e_2 else e_3: invert the lifted rule of the frame, apply the I.H., and close with the context rule of the frame.

□

Lemma C.13. (Well typed operations are handled) If $\cdot \vdash E\big[\texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s\ v\big] : \tau \mid \langle\rangle$ then $E = E_1[\texttt{handle}\ h\ e^{\varepsilon}]$ with $\texttt{op} \notin \texttt{bop}(E_2)$ and $(\texttt{op} \to f) \in h$.

Proof. Let $\texttt{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l)$.

$\cdot \vdash \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s\ v :$ Lemma C.12

$\tau_3 \mid \langle \lceil E \rceil \mid \langle\rangle \rangle$
$\cdot \vdash \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s :$ T-E-App

$\tau_4 \to^{\langle \lceil E \rceil \mid \langle\rangle \rangle}$

$\tau_3 \mid \langle \lceil E \rceil \mid \langle\rangle \rangle$
$\cdot \vdash_{\text{val}} \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s :$ T-E-Val

$\tau_4 \to^{\langle \lceil E \rceil \mid \langle\rangle \rangle} \tau_3$
$\langle l \mid \varepsilon_0 \rangle =$ T-E-Perform, where ε_0 is the explicit annotation of $\texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s$

$\langle \lceil E \rceil \mid \langle\rangle \rangle$
$l \in \lceil E \rceil$ the row is not empty

So some handler in E is for l, and it has a clause for op. Take $\texttt{handle}\ h\ e^{\varepsilon}$ to be the innermost handler of E with a clause for op. Then $(\texttt{op} \to f) \in h$, and no handler in E_2 handles op, so $\texttt{op} \notin \texttt{bop}(E_2)$. □

Lemma C.14. (Effect types are meaningful) If $\cdot \vdash E\big[\texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s\ v\big] : \tau \mid \varepsilon$ with $\texttt{op} \notin \texttt{bop}(E)$ and $\texttt{op} \in \Sigma(l)$, then $l \in \varepsilon$.

Proof.

$\cdot \vdash \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s\ v : \tau_3 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$ Lemma C.12
$\cdot \vdash_{\text{val}} \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_s : \tau_4 \to^{\langle \lceil E \rceil \mid \varepsilon \rangle} \tau_3$ T-E-App, T-E-Val
$\langle l \mid \varepsilon_0 \rangle = \langle \lceil E \rceil \mid \varepsilon \rangle$ T-E-Perform
$l \notin \lceil E \rceil$ $\texttt{op} \notin \texttt{bop}(E)$

The row on the left starts with l. If $\lceil E \rceil$ were not empty, the row on the right would start with a label of $\lceil E \rceil$, which is not l. So $\lceil E \rceil$ is empty and $\langle l \mid \varepsilon_0 \rangle = \varepsilon$, hence $l \in \varepsilon$. □

Lemma C.15. (Value typing) If $\Gamma \vdash v : \tau \mid \varepsilon$ then $\Gamma \vdash_{\text{val}} v : \tau$. Hence $\Gamma \vdash v : \tau \mid \varepsilon_0$ for every ε_0.

Proof. By induction on v. The second part follows from the first by T-E-Val.

- Case $v = \langle v_1, v_2 \rangle$. The derivation ends in T-E-Val or in the lifted T-Tuple. In the first case we are done. In the second:

  $\Gamma \vdash v_1 : \tau_1 \mid \varepsilon, \Gamma \vdash v_2 : \tau_2 \mid \varepsilon$ — `T-Tuple`
  $\Gamma \vdash_{\text{val}} v_1 : \tau_1, \Gamma \vdash_{\text{val}} v_2 : \tau_2$ — I.H.
  $\Gamma \vdash_{\text{val}} \langle v_1, v_2 \rangle : \langle \tau_1, \tau_2 \rangle$ — `T-E-VTuple`

- Any other value. The only rule that types it with an effect is T-E-Val, and its premise is the goal.

□

Lemma C.16. (Weakening) If $\Gamma \vdash e : \tau \mid \varepsilon$ and $x \notin \text{dom}(\Gamma)$ then $\Gamma, x : \tau_s \vdash e : \tau \mid \varepsilon$, and likewise for $\vdash_{\text{val}}$, $\vdash_{\text{ops}}$ and $\vdash_{\text{ec}}$.

Proof. By induction on the derivation. Only T-E-Abs and T-E-Fix extend Γ, with a bound variable that we choose different from x. The side condition $\alpha \notin \text{ftv}(\Gamma, x : \tau_s)$ of T-E-Perform holds by choosing the quantified α of the signature fresh. □

Lemma C.17. (Variable substitution) Let $\Gamma_1, \Gamma_2 \vdash_{\text{val}} v : \tau_1$.
1. If $\Gamma_1, x : \tau_1, \Gamma_2 \vdash e : \tau \mid \varepsilon$ then $\Gamma_1, \Gamma_2 \vdash e[x := v] : \tau \mid \varepsilon$.
2. If $\Gamma_1, x : \tau_1, \Gamma_2 \vdash_{\text{val}} v_0 : \tau$ then $\Gamma_1, \Gamma_2 \vdash_{\text{val}} v_0[x := v] : \tau$.
3. If $\Gamma_1, x : \tau_1, \Gamma_2 \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$ then $\Gamma_1, \Gamma_2 \vdash_{\text{ops}} h[x := v] : \tau \mid l \mid \varepsilon$.

Proof. By mutual induction on the derivations. Let $\Gamma = \Gamma_1, x : \tau_1, \Gamma_2$ and $\Gamma_0 = \Gamma_1, \Gamma_2$. Since v is a value, $v_0[x := v]$ is again a value, so the value rules still apply after substitution.

Part 2.

- Case T-E-Var with $v_0 = x$. Then $\tau = \tau_1$ and $x[x := v] = v$, which has type τ_1 by assumption.
- Case T-E-Var with $v_0 = y \neq x$.

  $y : \tau \in \Gamma_0$ — `T-E-Var`, $y \neq x$
  $\Gamma_0 \vdash_{\text{val}} y[x := v] : \tau$ — `T-E-Var`, $y[x := v] = y$

- Case T-E-Abs with $v_0 = \mathsf{lambda}^{\varepsilon}_0 y : \tau_2.e$, $y \notin \{x\} \cup \mathrm{fv}(v)$.

  $\Gamma, y : \tau_2 \vdash e : \tau_3 \mid \varepsilon_0$ — T-E-Abs
  $\Gamma_0, y : \tau_2 \vdash_{\mathrm{val}} v : \tau_1$ — Lemma C.16
  $\Gamma_0, y : \tau_2 \vdash e[x := v] : \tau_3 \mid \varepsilon_0$ — I.H. 1
  $\Gamma_0 \vdash_{\mathrm{val}} \mathsf{lambda}^{\varepsilon}_0 y : \tau_2.e[x := v] : \tau_2 \to^{\varepsilon_0} \tau_3$ — T-E-Abs

- Case T-E-Fix with $v_0 = \mu y : \tau.v_1$. As T-E-Abs, with the I.H. 2 on v_1.

- Case T-E-TAbs with $v_0 = \Lambda\alpha^k.v_1$.

  $\Gamma \vdash_{\mathrm{val}} v_1 : \tau_2, k \neq \mathtt{lab}$ — T-E-TAbs
  $\Gamma_0 \vdash_{\mathrm{val}} v_1[x := v] : \tau_2$ — I.H. 2
  $\Gamma_0 \vdash_{\mathrm{val}} \Lambda\alpha^k.v_1[x := v] : \forall\alpha^k.\tau_2$ — T-E-TAbs

- Case T-E-Handler with $v_0 = \mathtt{handler}\ h^{\varepsilon_0}$.

  $\Gamma \vdash_{\mathrm{ops}} h : \tau_2 \mid l \mid \varepsilon_0$ — T-E-Handler
  $\Gamma_0 \vdash_{\mathrm{ops}} h[x := v] : \tau_2 \mid l \mid \varepsilon_0$ — I.H. 3
  $\Gamma_0 \vdash_{\mathrm{val}} \mathtt{handler}\ h^{\varepsilon_0} : \left(\mathsf{unit} \to^{\langle l \mid \varepsilon_0 \rangle} \tau_2\right) \to^{\varepsilon_0} \tau_2$ — T-E-Handler

- Case T-E-VTuple with $v_0 = \langle v_1, v_2 \rangle$. The I.H. 2 on both components, then T-E-VTuple.

- Case T-E-Perform with v_0 an effect operation form. It has no **term** variables, so term substitution leaves its type instantiation and explicit tail annotation unchanged. The side condition $\alpha \notin \mathrm{ftv}(\Gamma_0)$ follows from $\alpha \notin \mathrm{ftv}(\Gamma)$.

- Cases T-Num, T-Bool, T-Unit, T-E-Error. No variables.

Part 1.

- Case T-E-Val.

  $\Gamma \vdash_{\mathrm{val}} v_0 : \tau$ — T-E-Val
  $\Gamma_0 \vdash_{\mathrm{val}} v_0[x := v] : \tau$ — I.H. 2
  $\Gamma_0 \vdash v_0[x := v] : \tau \mid \varepsilon$ — T-E-Val

- Case T-E-App with $e = e_1\ e_2$.

  $\Gamma \vdash e_1 : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon, \Gamma \vdash e_2 : \tau_2 \mid \varepsilon$ — T-E-App
  $\Gamma_0 \vdash e_1[x := v] : \tau_2 \to^{\varepsilon} \tau \mid \varepsilon$ — I.H. 1
  $\Gamma_0 \vdash e_2[x := v] : \tau_2 \mid \varepsilon$ — I.H. 1
  $\Gamma_0 \vdash e_1\ e_2[x := v] : \tau \mid \varepsilon$ — T-E-App

- Case T-E-Tapp with $e = e_1[\tau_0]$. The I.H. 1 on e_1, then T-E-Tapp.

- Case T-E-Handle with $e = \mathtt{handle}\ h\ e^{\varepsilon}$. The explicit residual annotation ε is unchanged by term substitution.

$\Gamma \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$ T-E-Handle
$\Gamma \vdash e_1 : \tau \mid \langle l \mid \varepsilon \rangle$ T-E-Handle
$\Gamma_0 \vdash_{\text{ops}} h[x := v] : \tau \mid l \mid \varepsilon$ I.H. 3
$\Gamma_0 \vdash e_1[x := v] : \tau \mid \langle l \mid \varepsilon \rangle$ I.H. 1
$\Gamma_0 \vdash (\texttt{handle}\ h\ e^{\varepsilon})[x := v] : \tau \mid \varepsilon$ T-E-Handle

- Cases of the lifted rules of Base (T-Tuple, T-If, T-Op, T-Zero, T-Inj-Left, T-Inj-Right). The I.H. 1 on every premise, then the same rule.

Part 3.

- Case T-E-Ops. Each operation clause has the type prescribed by T-E-Ops, with the residual row ε on all three arrows. Apply I.H. 2 to every clause body. The result has the same clause type under Γ_0, and T-E-Ops applies because its side conditions mention only ε and τ.

□

Lemma C.18. (Type variable substitution) If $\Gamma \vdash e : \tau \mid \varepsilon$ and $\vdash_{\text{wf}} \tau_0 : k$ then $\Gamma[\alpha^k := \tau_0] \vdash e[\alpha^k := \tau_0] : \tau[\alpha^k := \tau_0] \mid \varepsilon[\alpha^k := \tau_0]$, and likewise for $\vdash_{\text{val}}$ and $\vdash_{\text{ops}}$.

Proof. By induction on the derivation, as Lemma 20 of [16] without the evidence translation. Bound type variables are renamed away from α and from $\text{ftv}(\tau_0)$, so substitution commutes with them:

$\tau_2[\beta := \tau_s][\alpha := \tau_0] = \tau_2[\alpha := \tau_0]\left[\beta := \tau_{s[\alpha := \tau_0]}\right]$ when $\beta \notin \{\alpha\} \cup \text{ftv}(\tau_0)$.

The interesting cases are the following.

- Case T-E-Tapp with $e = e_1[\tau_s]$.

$\Gamma \vdash e_1 : \forall \beta.\tau_2 \mid \varepsilon$ T-E-Tapp
$\Gamma[\alpha := \tau_0] \vdash e_1[\alpha := \tau_0] : \forall \beta.\tau_2[\alpha := \tau_0] \mid \varepsilon[\alpha := \tau_0]$ I.H.
$\Gamma[\alpha := \tau_0] \vdash e_1[\alpha := \tau_0]\left[\tau_{s[\alpha := \tau_0]}\right] : \tau_2[\alpha := \tau_0]\left[\beta := \tau_{s[\alpha := \tau_0]}\right] \mid \varepsilon[\alpha := \tau_0]$ T-E-Tapp
$\tau_2[\alpha := \tau_0]\left[\beta := \tau_{s[\alpha := \tau_0]}\right] = \tau_2[\beta := \tau_s][\alpha := \tau_0]$ commuting

- Case T-E-Perform. The type of the operation in $\Sigma(l)$ is closed, so only the instantiation τ_s changes, and the conclusion follows by commuting. The quantified variable of the signature is chosen fresh, so it is still not free in $\Gamma[\alpha := \tau_0]$.
- Case T-E-Ops. The clause variable is chosen fresh for τ_0, so the side condition $\beta \notin \text{ftv}(\varepsilon[\alpha := \tau_0], \tau[\alpha := \tau_0])$ still holds.
- Case T-E-TAbs. The bound variable is renamed, and $k \neq \texttt{lab}$ is unchanged.

All other cases follow from the I.H. and the same rule. □

Lemma C.19. (Canonical forms) Let $\cdot \vdash v : \tau \mid \varepsilon$. Then v is $\texttt{error}_p$, or

- if $\tau = \tau_1 \to^{\varepsilon_0} \tau_2$, v is $\mathsf{lambda}^{\varepsilon_0} x : \tau_1.e$, an operation form $\texttt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s$ whose operation row is ε_0, a handler form $\texttt{handler}\ h^{\varepsilon_0}$, or $\mu x : \tau.v_0$;
- if $\tau = \forall \alpha.\tau_s$, v is $\Lambda\alpha.v_0$ or $\mu x : \tau.v_0$;
- if τ is num, bool, unit or $\langle \tau_1, \tau_2 \rangle$, v is n, b, unit or $\langle v_1, v_2 \rangle$ respectively.

Proof. A variable is not typed in the empty context. By Lemma C.15, $\cdot \vdash_{\mathrm{val}} v : \tau$. Every other value form has exactly one value rule, and the conclusion of that rule fixes the shape of the type. T-E-Error types $\texttt{error}_p$ at any type, and T-E-Fix only gives a function or polymorphic type. □

Lemma C.20. (Steps under contexts) If $e \Longrightarrow e_0$ then $E[e] \Longrightarrow e_0$ for some e_0.

Proof. If the step is Step-E, $e = E_1[e_1]$ with $e_1 \blacktriangleright e_2$, and $E[e] = (E[E_1])[e_1] \Longrightarrow (E[E_1])[e_2]$ by Step-E. If it is Step-E-Error, $e = E_1\left[\texttt{error}_p\right]$ with $E_1 \neq \square$, so $E[E_1] \neq \square$ and $E[e] \Longrightarrow \texttt{error}_p$ by Step-E-Error. □

Lemma C.21. (Progress with effects) If $\cdot \vdash e_1 : \tau \mid \varepsilon$ then e_1 is a value, or $e_1 \Longrightarrow e_2$, or $e_1 = E\left[\texttt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right]$ with $\mathtt{op} \notin \texttt{bop}(E)$.

Proof. By induction on the typing derivation. Call the last form *stuck on op*.

- Case T-E-Val. e_1 is a value.
- Case T-E-App, $e_1 = e_3\ e_4$.

  $\cdot \vdash e_3 : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App

  $\cdot \vdash e_4 : \tau_1 \mid \varepsilon$ T-E-App

  By the I.H. on e_3:
  - $e_3 \Longrightarrow e_5$. Then e_1 steps by Lemma C.20.
  - $e_3 = E_0\left[\texttt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right]$. Take $E = E_0\ e_4$. Then $\texttt{bop}(E) = \texttt{bop}(E_0)$, so e_1 is stuck on $\mathtt{op}$.
  - e_3 is a value. By the I.H. on e_4:
    - $e_4 \Longrightarrow e_6$. Then e_1 steps by Lemma C.20.
    - $e_4 = E_0\left[\texttt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right]$. Take $E = e_3\ E_0$.
    - e_4 is a value. By Lemma C.19 on e_3:
      - $e_3 = \mathsf{lambda}^{\varepsilon} x : \tau_1.e$. Then $e_1 \Longrightarrow e[x := e_4]$ (S-App).
      - $e_3 = \texttt{handler}\ h^{\varepsilon}$. Then $e_1 \Longrightarrow \texttt{handle}\ h\ e^{\varepsilon}$ (Step-E-Handler).
      - $e_3 = \texttt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s$. Take $E = \square$; e_1 is stuck on $\mathtt{op}$.
      - $e_3 = \mu x : \tau_f.v_0$. Then $e_1 \Longrightarrow v_0[x := e_3]\ e_4$ (Step-E-Fix-App).
      - $e_3 = \texttt{error}_p$. Error subcase, with the context $\square\ e_4$.

- Case T-E-Tapp, $e_1 = e_3[\tau_1]$ and $\cdot \vdash e_3 : \forall\alpha.\tau_2 \mid \varepsilon$. By the I.H. on e_3:
  - e_3 steps, or is stuck on $\mathtt{op}$ with context E_0. Then e_1 steps, or is stuck on $\mathtt{op}$ with context $E_0[\tau_1]$.
  - e_3 is a value. By Lemma C.19:
    - $e_3 = \Lambda\alpha.v_0$. Then $e_1 \Longrightarrow v_0[\alpha := \tau_1]$ (Step-E-TApp).
    - $e_3 = \mu x : \tau_f.v_0$. Then $e_1 \Longrightarrow v_0[x := e_3][\tau_1]$ (Step-E-Fix-TApp).
    - $e_3 = \mathtt{error}_p$. Error subcase.
- Case T-E-Handle, $e_1 = \mathtt{handle}\ h\ e^{\varepsilon}$ with $h : \Sigma(l)$ and $\cdot \vdash e_3 : \tau \mid \langle l \mid \varepsilon \rangle$. By the I.H. on e_3:
  - e_3 steps. Then e_1 steps by Lemma C.20.
  - e_3 is a value. Then $e_1 \Longrightarrow e_3$ (Step-E-Return).
  - $e_3 = E_0\left[\mathtt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right]$ with $\mathtt{op} \notin \mathtt{bop}(E_0)$.
    - If $(\mathtt{op} \to f) \in h$, then $e_1 \Longrightarrow f[\tau_s]\, v\, k$ with $k = \mathsf{lambda}^{\varepsilon} x : \tau_2[\alpha := \tau_s].\mathtt{handle}\ h\ e^{\varepsilon}$ (Step-E-Perform).
    - Otherwise take $E = \mathtt{handle}\ h\ e^{\varepsilon}$. Then $\mathtt{op} \notin \mathtt{bop}(E) = \mathtt{bop}(E_0) \cup \{\mathtt{op}_0 \mid (\mathtt{op}_0 \to f) \in h\}$, so e_1 is stuck on $\mathtt{op}$.
- Case T-If, $e_1 = \mathsf{if}\ e_3\ \mathsf{then}\ e_4\ \mathsf{else}\ e_5$. If e_3 steps or is stuck, so is e_1, with the context $\mathsf{if}\ E_0\ \mathsf{then}\ e_4\ \mathsf{else}\ e_5$. If e_3 is a value of type bool, it is true or false and S-If-True or S-If-False applies, or it is $\mathtt{error}_p$ (error subcase).
- Cases T-Op and T-Zero. As T-E-App: evaluate the operands from left to right; if all are values they are numbers or booleans by Lemma C.19 and S-Op or S-Zero applies, or one of them is $\mathtt{error}_p$ (error subcase).
- Cases T-Tuple, T-Inj-Left and T-Inj-Right. If the operands are values, a tuple is a value; $\mathsf{inj.l}(\langle v_1, v_2 \rangle)$ and $\mathsf{inj.r}(\langle v_1, v_2 \rangle)$ step by S-Inj-Left and S-Inj-Right; an operand $\mathtt{error}_p$ is the error subcase.

□

Theorem C.22. ($\mathtt{Effects}$ Progress) If $\cdot \vdash e_1 : \tau \mid \langle\rangle$ then e_1 is a value or $e_1 \Longrightarrow e_2$.

Proof. By Lemma C.21, e_1 is a value, or steps, or $e_1 = E\left[\mathtt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right]$ with $\mathtt{op} \notin \mathtt{bop}(E)$ and $\mathtt{op} \in \Sigma(l)$. The last case is impossible:

$\cdot \vdash E\left[\mathtt{perform}\ \varepsilon_p\ \mathtt{op}\ \tau_s\ v\right] : \tau \mid \langle\rangle$	Given
$l \in \langle\rangle$	Lemma C.14
Contradiction	$\langle\rangle$ is empty

□

Lemma C.23. (Redex preservation) If $\cdot \vdash e_1 : \tau \mid \varepsilon$ and $e_1 \rightarrow e_2$ then $\cdot \vdash e_2 : \tau \mid \varepsilon$.

Proof. By cases on $e_1 \rightarrow e_2$.

- Case S-App, $(\mathsf{lambda}^{\varepsilon} x : \tau_1.e)\ v \rightarrow e[x := v]$.

  $\cdot \vdash \mathsf{lambda}^{\varepsilon} x : \tau_1.e : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App
  The λ annotation is the body effect ε. T-E-Abs
  $\cdot \vdash v : \tau_1 \mid \varepsilon$ T-E-App
  $x : \tau_1 \vdash e : \tau \mid \varepsilon$ T-E-Val, T-E-Abs
  $\cdot \vdash_{\text{val}} v : \tau_1$ Lemma C.15
  $\cdot \vdash e[x := v] : \tau \mid \varepsilon$ Lemma C.17

- Case Step-E-TApp, $(\Lambda\alpha^k.v)[\tau_0] \rightarrow v[\alpha := \tau_0]$, with $\tau = \tau_1[\alpha := \tau_0]$.

  $\cdot \vdash \Lambda\alpha^k.v : \forall\alpha^k.\tau_1 \mid \varepsilon, \vdash_{\text{wf}} \tau_0 : k$ T-E-Tapp
  $\cdot \vdash_{\text{val}} v : \tau_1$ T-E-Val, T-E-TAbs
  $\cdot \vdash_{\text{val}} v[\alpha := \tau_0] : \tau_1[\alpha := \tau_0]$ Lemma C.18
  $\cdot \vdash v[\alpha := \tau_0] : \tau \mid \varepsilon$ T-E-Val

  The context is empty, and α is bound in $\Lambda\alpha.v$, so $\alpha \notin \text{ftv}(\varepsilon)$ and $\varepsilon[\alpha := \tau_0] = \varepsilon$.

- Case Step-E-Fix-App, $(\mu x : \tau_f.v)\ v_0 \rightarrow v\left[x := \mu x : \tau_f.v\right]\ v_0$, with $\tau_f = \tau_1 \to^{\varepsilon} \tau$.

  $\cdot \vdash_{\text{val}} \mu x : \tau_f.v : \tau_f$ T-E-App, T-E-Val
  $\cdot \vdash v_0 : \tau_1 \mid \varepsilon$ T-E-App
  $x : \tau_f \vdash_{\text{val}} v : \tau_f$ T-E-Fix
  $\cdot \vdash_{\text{val}} v\left[x := \mu x : \tau_f.v\right] : \tau_f$ Lemma C.17
  $\cdot \vdash v\left[x := \mu x : \tau_f.v\right]\ v_0 : \tau \mid \varepsilon$ T-E-Val, T-E-App

- Case Step-E-Fix-TApp. As Step-E-Fix-App, closing with T-E-Tapp.

- Case Step-E-Handler, $\texttt{handler}\ h^{\varepsilon}\ v \rightarrow \texttt{handle}\ h\ e^{\varepsilon}$.

  $\cdot \vdash \texttt{handler}\ h^{\varepsilon} : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App
  $\cdot \vdash v : \tau_1 \mid \varepsilon$ T-E-App
  $\cdot \vdash_{\text{val}} \texttt{handler}\ h^{\varepsilon} : \left(\mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau\right) \to^{\varepsilon} \tau$ T-E-Val
  $\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon, \tau_1 = \mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau$ T-E-Handler
  $\cdot \vdash v : \mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau \mid \langle l \mid \varepsilon \rangle$ Lemma C.15
  $\cdot \vdash v\ \mathsf{unit} : \tau \mid \langle l \mid \varepsilon \rangle$ T-Unit, T-E-App
  $\cdot \vdash \texttt{handle}\ h\ e^{\varepsilon} : \tau \mid \varepsilon$ T-E-Handle

- Case Step-E-Return, $\texttt{handle}\ h\ e^{\varepsilon} \rightarrow v$.

  $\cdot \vdash v : \tau \mid \langle l \mid \varepsilon \rangle$ T-E-Handle
  $\cdot \vdash v : \tau \mid \varepsilon$ Lemma C.15

- Case Step-E-Perform, $\texttt{handle}\ h\ e^{\varepsilon} \rightarrow f[\tau_0]\ v\ k$, where $\texttt{op} \notin \texttt{bop}(E)$, $(\texttt{op} \to f) \in h$, $\texttt{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l)$ and $k = \mathsf{lambda}^{\varepsilon} x : \tau_2[\alpha := \tau_0].\texttt{handle}\ h\ e^{\varepsilon}$.

$\cdot \vdash_{\text{ec}} \texttt{handle}\ h\ e^{\varepsilon} : \tau_3 \to \tau \mid \varepsilon$	Lemma C.12
$\cdot \vdash \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_0\ v : \tau_3 \mid \langle \lceil E \rceil \mid \langle l \mid \varepsilon \rangle \rangle$	Lemma C.12
$\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon, \cdot \vdash_{\text{ec}} E : \tau_3 \to \tau \mid \langle l \mid \varepsilon \rangle$	CE-Handle
$\cdot \vdash_{\text{val}} f : \forall \alpha . \tau_1 \to^{\varepsilon} ((\tau_2 \to^{\varepsilon} \tau) \to^{\varepsilon} \tau)$	T-E-Ops
$\alpha \notin \text{ftv}(\varepsilon, \tau)$	T-E-Ops
$\cdot \vdash f[\tau_0] : \tau_1[\alpha := \tau_0] \to^{\varepsilon} ((\tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau) \to^{\varepsilon} \tau) \mid \varepsilon$	T-E-Val, T-E-Tapp
$\cdot \vdash_{\text{val}} \texttt{perform}\ \varepsilon_p\ \texttt{op}\ \tau_0 : \tau_4 \to^{\langle \lceil E \rceil \mid \langle l \mid \varepsilon \rangle \rangle} \tau_3$	T-E-App, T-E-Val
$\tau_4 = \tau_1[\alpha := \tau_0], \tau_3 = \tau_2[\alpha := \tau_0]$	T-E-Perform
$\cdot \vdash v : \tau_1[\alpha := \tau_0] \mid \varepsilon$	T-E-App, Lemma C.15
$\cdot \vdash f[\tau_0]\ v : (\tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau) \to^{\varepsilon} \tau \mid \varepsilon$	T-E-App
$x : \tau_2[\alpha := \tau_0] \vdash x : \tau_2[\alpha := \tau_0] \mid \langle \lceil E \rceil \mid \langle l \mid \varepsilon \rangle \rangle$	T-E-Var, T-E-Val
$x : \tau_2[\alpha := \tau_0] \vdash_{\text{ec}} \texttt{handle}\ h\ e^{\varepsilon} : \tau_2[\alpha := \tau_0] \to \tau \mid \varepsilon$	Lemma C.16
$x : \tau_2[\alpha := \tau_0] \vdash \texttt{handle}\ h\ e^{\varepsilon} : \tau \mid \varepsilon$	Lemma C.11
$\cdot \vdash k : \tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau \mid \varepsilon$	T-E-Abs, T-E-Val
$\cdot \vdash f[\tau_0]\ v\ k : \tau \mid \varepsilon$	T-E-App

Substituting for α leaves ε and τ unchanged, because $\alpha \notin \text{ftv}(\varepsilon, \tau)$.

- Cases S-If-True, S-If-False, S-Inj-Left, S-Inj-Right, S-Op, S-Zero. As in Lemma A.6, with every judgment at effect ε.

□

Theorem C.24. (Effects Preservation) If $\cdot \vdash e_1 : \tau \mid \varepsilon$ and $e_1 \Longrightarrow e_2$ then $\cdot \vdash e_2 : \tau \mid \varepsilon$.

Proof. By cases on the step.

- Case Step-E, $e_1 = E[e_a]$, $e_2 = E[e_b]$ and $e_a \rightarrow e_b$.

$\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$	Lemma C.12
$\cdot \vdash e_a : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$	Lemma C.12
$\cdot \vdash e_b : \tau_1 \mid \langle \lceil E \rceil \mid \varepsilon \rangle$	Lemma C.23
$\cdot \vdash E[e_b] : \tau \mid \varepsilon$	Lemma C.11

- Case Step-E-Error, $E[\texttt{error}_p] \Longrightarrow \texttt{error}_p$.

$\cdot \vdash_{\text{val}} \texttt{error}_p : \tau$	T-E-Error
$\cdot \vdash \texttt{error}_p : \tau \mid \varepsilon$	T-E-Val

□

Theorem C.25. (Effects Type Safety) If $\cdot \vdash e : \tau \mid \langle\rangle$ and $e \Longrightarrow^* e_0$ then e_0 is a value or $e_0 \Longrightarrow e_0$ for some e_0.

Proof. By Theorem C.24 on each step, $\cdot \vdash e_0 : \tau \mid \langle\rangle$. Then by Theorem C.22. □

CHAPTER D

Compiler Typesoundness Proof

D.1 Overview

This part shows how the compiler remains typesoundness of the source program to the target program. Since the compiler introduces effects and explicitly declares it everywhere (in lambdas, and perform operations), the function type is augmented with the effect. The compiler also does not introduce any handler, this makes the whole program (potentially) produce the effect as well.

The environment that will be used to type check target programs has the following Σ signature.

$$\Sigma(\mathtt{h}_{\mathscr{C}}) = \left\{\mathscr{C} : \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \alpha\rangle\rangle \to \alpha\right\}$$

D.2 Type Translation

$\cdot^* : \mathsf{Contracts}\ \tau \to \mathtt{Effects}\ \tau$

$$\begin{aligned}
\mathsf{num}^* &= \mathsf{num} \\
\mathsf{bool}^* &= \mathsf{bool} \\
\mathsf{unit}^* &= \mathsf{unit} \\
\langle \tau_1, \tau_2 \rangle^* &= \langle \tau_1^*, \tau_2^* \rangle \\
(\tau_1 \to \tau_2)^* &= \tau_1^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^* \\
(x_1 : \tau_1, ..., x_n : \tau_n)^* &= x_1 : \tau_1^*, ..., x_n : \tau_n^*
\end{aligned}$$

D.3 Compiler Type Preservation

Theorem D.1. (Compiler Type Preservation)

If $\Gamma \vdash e : \tau \rightarrowtail e'$ then $\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$.

Proof.

By induction on structure of $\Gamma \vdash e : \tau \rightarrowtail e'$.

- Case $e = x$, $e = n$, $e = b$, $e = \mathsf{unit}$

  $e' = e$ — Given

  $\Gamma^* \vdash_{\mathrm{val}} e' : \tau^*$ — T-E-Var, T-Num, T-Bool, T-Unit

  $\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — T-E-Val

- Case

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \rightarrowtail e''}{\Gamma \vdash \lambda x : \tau_1 . e : \tau_1 \to \tau_2 \rightarrowtail \lambda x : \tau_1^* . e''}$$

  $\Gamma^*, x : \tau_1^* \vdash e'' : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — I.H.

  $\Gamma^* \vdash_{\mathrm{val}} \lambda x : \tau_1^* . e'' : \tau_1^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^*$ — T-E-Abs

  $\Gamma^* \vdash_{\mathrm{val}} \lambda x : \tau_1^* . e'' : (\tau_1 \to \tau_2)^*$ — Type compilation $\cdot^*$

  $\Gamma^* \vdash \lambda x : \tau_1^* . e'' : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — T-E-Val

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail e_1' \qquad \Gamma \vdash e_2 : \tau_1 \rightarrowtail e_2'}{\Gamma \vdash e_1\ e_2 : \tau_2 \rightarrowtail e_1'\ e_2'}$$

  $\Gamma^* \vdash e_1' : \tau_1^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — I.H.

  $\Gamma^* \vdash e_2' : \tau_1^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — I.H.

  $\Gamma^* \vdash e_1'\ e_2' : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — T-E-App

  The effect on the arrow and the effect of the premises are both $\langle \mathtt{h}_{\mathscr{C}} \rangle$.

- Case $\mu x : \tau . e$, $\langle e_1, e_2 \rangle$, $\mathsf{inj.l}(e)$, $\mathsf{inj.r}(e)$, if e_1 then e_2 else e_3, $o(e_1, ..., e_n)$

  $\Gamma^* \vdash e_i' : \tau_i^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ for every sub-expression — I.H.

  $\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — lifted rule of Base

  For $\mu x : \tau . e$, the binder is $x : \tau^*$, and the I.H. is used under $\Gamma^*, x : \tau^*$.

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau^*\ \langle k, \langle e_5, e_6 \rangle \rangle}$$

  $\Gamma^* \vdash e_5 : \tau^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — I.H.

$\Gamma^* \vdash e_6 : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^* \vdash \mathtt{perform}\ \mathscr{C}\ \tau^*\ \langle k, \langle e_5, e_6 \rangle\rangle : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ Lemma D.2

This is the only rule that introduces $\mathtt{h}_{\mathscr{C}}$.

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail e_2 \qquad \Gamma \vdash \lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}$$

$\Gamma^* \vdash e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^* \vdash e_3 : ((\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2))^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^* \vdash e_3 : (\tau_1 \to \tau_2)^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ def. of $\cdot^*$

$\Gamma^* \vdash e_3\ e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail e_2 \qquad \Gamma \vdash e_w : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}$$

where $e_w = \lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\Big(\Big\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\Big\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$\Gamma^* \vdash e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^* \vdash e_3 : (\tau_1 \to \tau_2)^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H., def. of $\cdot^*$

$\Gamma^* \vdash e_3\ e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

The dependency y is replaced in the source by the indy monitor $\mathsf{mon}_j^{l,j}(\kappa_1, x) : \tau_1$, so the I.H. on e_w already covers κ_2 in the context $\Gamma, f : \tau_1 \to \tau_2, x : \tau_1$.

- Case

$$\frac{x \notin \mathrm{dom}(\Gamma) \qquad \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_1, \mathsf{inj.l}(x)) : \tau_1 \rightarrowtail e_1 \qquad \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_2, \mathsf{inj.r}(x)) : \tau_2 \rightarrowtail e_2 \qquad \Gamma \vdash e : \langle \tau_1, \tau_2 \rangle \rightarrowtail e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\langle \kappa_1, \kappa_2 \rangle, e) : \langle \tau_1, \tau_2 \rangle \rightarrowtail (\lambda x : \langle \tau_1, \tau_2 \rangle^* . \langle e_1, e_2 \rangle)\ e_3}$$

$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash e_1 : \tau_1^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash e_2 : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-Tuple

$\Gamma^* \vdash_{\mathrm{val}} \lambda x : \langle \tau_1, \tau_2 \rangle^* . \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \langle \tau_1, \tau_2 \rangle^*$ T-E-Abs

$\Gamma^* \vdash \lambda x : \langle \tau_1, \tau_2 \rangle^* . \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-Val

$\Gamma^* \vdash e_3 : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.

$\Gamma^* \vdash (\lambda x . \langle e_1, e_2 \rangle)\ e_3 : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

□

Lemma D.2. (Typing a perform of the check effect)

If $\Gamma \vdash e_1 : \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ and $\Gamma \vdash e_2 : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ then

$\Gamma \vdash \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_1, e_2 \rangle\rangle : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$.

Proof.

$\Gamma \vdash_{\mathrm{val}} \mathtt{perform}\ \mathscr{C}\ \tau : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle\rangle \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau$	T-E-Perform, $\alpha := \tau, \varepsilon := \langle\rangle$
$\Gamma \vdash \mathtt{perform}\ \mathscr{C}\ \tau : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle\rangle \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-E-Val
$\Gamma \vdash k : \mathtt{blab} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-E-Label
$\Gamma \vdash e_1 : \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	Given
$\Gamma \vdash e_2 : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	Given
$\Gamma \vdash \langle e_1, e_2 \rangle : \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-Tuple
$\Gamma \vdash \langle k, \langle e_1, e_2 \rangle\rangle : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle\rangle \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-Tuple
$\Gamma \vdash \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_1, e_2 \rangle\rangle : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-E-App

In the last step, the effect on the arrow and the effect of both premises are the same row $\langle \mathtt{h}_{\mathscr{C}} \rangle$. □

D.4 Wrapped Compiled Program Typing

The deep handler wrapper from the compiler equivalence proof removes $\mathtt{h}_{\mathscr{C}}$ from the effect row. We rewrite the wrapper as follows:

$$\mathsf{wrap} = \mu w : \forall \alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha.\Lambda\alpha.\lambda f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha.\ \mathtt{handler}\ h_w\ f$$

$$h_w = \big\{\mathscr{C} \to \Lambda\beta.\lambda\langle l, \langle e, v\rangle\rangle.\lambda k : \beta \to^{\langle\rangle} \alpha.\ \text{if } w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v)\ \text{then } k\ v\ \text{else } \mathtt{error}_l\big\}$$

Note that h_w signature matches $\Sigma(\mathtt{h}_{\mathscr{C}})$ if we replace α with β.

Theorem D.3. (Compiler Effect Safety)

If $\cdot \vdash e : \tau \rightarrowtail e'$ then $\cdot \vdash \mathsf{wrap}[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$.

Proof.

$\cdot \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	Theorem D.1
$\cdot \vdash_{\mathrm{val}} \lambda_ : \mathsf{unit}.e' : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau^*$	T-E-Abs
$\cdot \vdash \mathsf{wrap}[\tau^*] : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau^*\big) \to^{\langle\rangle} \tau^* \mid \langle\rangle$	Lemma D.4, T-E-Tapp
$\cdot \vdash \mathsf{wrap}[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$	T-E-App

□

Lemma D.4. (Type of wrap)

$\cdot \vdash_{\text{val}} \mathsf{wrap} : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$

Proof.

Assume contexts $\Gamma_w = w : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$.

And $\Delta = \Gamma_w, l : \mathtt{blab}, e : \beta \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, v : \beta, k : \beta \to^{\langle\rangle} \alpha$.

$\Delta \vdash e\ v : \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$	T-E-App
$\Delta \vdash_{\text{val}} \lambda_ : \mathsf{unit}.e\ v : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}$	T-E-Abs
$\Delta \vdash w[\mathsf{bool}] : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}\big) \to^{\langle\rangle} \mathsf{bool} \mid \langle\rangle$	T-E-Tapp
$\Delta \vdash w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v) : \mathsf{bool} \mid \langle\rangle$	T-E-App
$\Delta \vdash k\ v : \alpha \mid \langle\rangle$	T-E-App
$\Delta \vdash \mathtt{error}_l : \alpha \mid \langle\rangle$	T-E-Error
$\Delta \vdash \mathsf{if}\ w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v)\ \mathsf{then}\ k\ v\ \mathsf{else}\ \mathtt{error}_l : \alpha \mid \langle\rangle$	T-If
$\Gamma_w \vdash_{\text{val}} h_{w(\mathscr{C})} : \forall\beta.\langle \mathtt{blab}, \langle \beta \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \beta \rangle \rangle \to^{\langle\rangle} \big(\big(\beta \to^{\langle\rangle} \alpha\big) \to^{\langle\rangle} \alpha\big)$	T-E-Abs, T-E-TAbs
$\Gamma_w \vdash_{\text{ops}} h_w : \alpha \mid \langle \mathtt{h}_{\mathscr{C}} \mid \langle\rangle \rangle$	T-E-Ops
$\Gamma_w \vdash_{\text{val}} \mathtt{handler}\ h_w : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$	T-E-Handler
$\Gamma_w, f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha \vdash \mathtt{handler}\ h_w\ f : \alpha \mid \langle\rangle$	T-E-App
$\Gamma_w \vdash_{\text{val}} \lambda f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha.\ \mathtt{handler}\ h_w\ f : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$	T-E-Abs
$\Gamma_w \vdash_{\text{val}} \Lambda\alpha.\lambda f.\ \mathtt{handler}\ h_w\ f : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$	T-E-TAbs
$\cdot \vdash_{\text{val}} \mathsf{wrap} : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$	T-Fix

□

CHAPTER E

Compiler Correctness Proof (Deep Handler)

E.1 Overview

Proof for compiler from Contracts to Untyped Deep `Effects`, the handler semantic is deep, where the continuation automatically reinstall the handler.

$\mathtt{h}_{\mathscr{C}} : \mathtt{lab} \quad = \quad \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\}$

$\mathsf{wrap} \quad = \quad \mu w : \forall\alpha.\big(\big(() \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big) \to \alpha\big).\Lambda\alpha.\, \lambda f : \big(() \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big).$

$\mathtt{handler}\ \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } w[\mathsf{bool}]\ (\lambda_.e\ v)\ \text{then } k\ v\ \text{else } \mathtt{error}_l\}\ f$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the argument as tuple pattern

The proof get into the way a lot because it switch back and forth between states prepended with handle and states without handle.

Compiler Correctness by proving (1) Contracts $\sim$ Untyped `Effects`, (2) Untyped `Effects` $\sim$ `Effects`, then obtain Contracts $\sim$ `Effects`. (2) is trivial.

E.2 Relation

Let's first define these relations:

- $R_e \subseteq$ Contracts $e \times$ Untyped `Effects` e

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $E \times$ Untyped `Effects` E

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

We define a separated context to deal with how $\mathsf{check}_j^k(e, v)$ are related from source to target.

$\sim \subseteq$ Contracts $E^+ \times$ Untyped `Effects` E^+

$E^+ = \square \mid E^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$

Define $\tilde{E}^+$ such that $E^+ \sim \tilde{E}^+$.

$$\begin{array}{rcl} \square & \sim & \square \\ E^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right] & \sim & \tilde{E}^+\left[\text{if } \square \text{ then } \left(\lambda x.\ \texttt{handle h}_{\mathscr{C}}\ \tilde{E}^*[x]\right)\tilde{v} \text{ else } \texttt{error}_j\right] \end{array}$$

$\sim \subseteq$ Contracts $E^* \times$ Untyped `Effects` E^*

E^* is a context without $\mathsf{check}_j^k(e, v)$ in Contracts.

Define $\tilde{E}^*$ such that $E^* \sim \tilde{E}^*$.

$$\begin{array}{rcl} \square & \sim & \square \\ E^*\ e & \sim & \tilde{E}^*\ \tilde{e} \\ v\ E^* & \sim & \tilde{v}\ \tilde{E}^* \\ \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E^*) & \sim & \texttt{perform}\ \mathscr{C}\ \langle \tilde{e}, \tilde{E}^* \rangle \\ \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\ & & e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) \text{ row below} \\ \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\ & & e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e) \text{ row below} \end{array}$$

$\sim \subseteq$ Contracts $e \times$ Untyped `Effects` e

E^* is a context without $\mathsf{check}_j^k(e, v)$ in Contracts. So it does not have the check cases below.

$$\begin{array}{rcl} v & \sim & \tilde{v} \\ e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\ \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \texttt{perform}\ \mathscr{C}\langle \tilde{e}_1, \tilde{e}_2 \rangle \\ \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\ & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \end{array}$$

$$\begin{array}{rcl}
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\big\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\big\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{blame}_j^k & \sim & \texttt{error}_j
\end{array}$$

$\sim \subseteq \mathsf{Contracts}\ v \times \text{Untyped}\ \texttt{Effects}\ v$

$$\begin{array}{rcl}
x & \sim & x \\
b & \sim & b \\
N & \sim & N \\
\lambda x.e & \sim & \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, \tilde{v}\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x.\mathsf{mon}_j^{k,l}\big(\big\{\mathsf{mon}_j^{l,j}(\kappa_1, \widetilde{x)/y}\big\}\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)
\end{array}$$

Lemma E.1. (Relation is correct) If $e \sim e'$ and $E \sim E'$ then $E[e] \sim E'[e']$.

Proof. By induction on the structure of E.

Base case: $E = \square$.

$E[e] = e \sim e' = E'[e']$ Assumption

Inductive cases:

- Case $E = E_1\ e_0$ and $E' = E_1'\ e_0'$ where $E_1 \sim E_1'$ and $e_0 \sim e_0'$.

  $E_1[e] \sim E_1'[e']$ I.H.
  $E_1[e]\ e_0 \sim E_1'[e']\ e_0'$ Relation definition

- Case $E = v\ E_1$ and $E' = v'\ E_1'$ where $v \sim v'$ and $E_1 \sim E_1'$.

  $E_1[e] \sim E_1'[e']$ I.H.
  $v\ E_1[e] \sim v'\ E_1'[e']$ Relation definition

- Case $E = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_0), E_1)$ and $E' = \texttt{perform}\ \mathscr{C}\ \langle e_0', E_1'\rangle$ where $e_0 \sim e_0'$ and $E_1 \sim E_1'$.

  $E_1[e] \sim E_1'[e']$ I.H.
  $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_0), E_1[e]) \sim \texttt{perform}\ \mathscr{C}\ \langle e_0', E_1'[e']\rangle$ Relation definition

- Case $E = \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E_1)$ and $E' = e_c'\ E_1'$ where $e_c \sim e_c'$ and $E_1 \sim E_1'$, with $e_c = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$.

$E_1[e] \sim E_1'[e']$	I.H.
$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E_1[e]) \sim e_c'\, E_1'[e']$	Relation row for $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e)$, with $e := E_1[e]$ and $\tilde{e}_1 := e_c'$

- Case $E = \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E_1)$ and $E' = e_c'\, E_1'$ where $e_c \sim e_c'$ and $E_1 \sim E_1'$, with $e_c = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\left\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\right\}\kappa_2, f\, \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$.

$E_1[e] \sim E_1'[e']$	I.H.
$\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E_1[e]) \sim e_c'\, E_1'[e']$	Relation row for $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e)$, with $e := E_1[e]$ and $\tilde{e}_1 := e_c'$

Note: We don't need explicit check cases here since E^* contexts don't contain checks by definition.

All cases follow from the induction hypothesis and the compositional nature of the relation. □

Lemma E.2. (Relation decomposition through check-free contexts)

If $E^*[e] \sim e'$, then there exist E'^* and e_0' such that $E^* \sim E'^*$ and $e \sim e_0'$ and $e' = E'^*[e_0']$.

Proof. □

Lemma E.3. (Relation is closed under substitution) If $e \sim e'$ and $v \sim v'$ then $e[x := v] \sim e'[x := v']$.

Proof. By induction on the derivation of $e \sim e'$.

- Case $x \sim x$.

$x[x := v] = v \sim v' = x[x := v']$	Assumption

- Case $z \sim z$ with $z \neq x$.

$z[x := v] = z \sim z = z[x := v']$	Both sides are unchanged

- Case $\lambda z.e_1 \sim \lambda z.e_1'$ with $e_1 \sim e_1'$. Choose $z \neq x$ and $z \notin \mathrm{fv}(v, v')$ by renaming.

$e_1[x := v] \sim e_1'[x := v']$	I.H.
$\lambda z.e_1[x := v] \sim \lambda z.e_1'[x := v']$	The λ row

- All other rows (`apply`, `mon` with flat, arrow and dependent contracts, `guard`, b, N, `blame`) are compositional. The substitution goes into every related sub-expression, including the predicates inside contracts and the wrappers e_1, and each follows by I.H.

□

Lemma E.4. (Breaking down Context E) For all E, $E = E^+[E^*]$

Proof. By induction on E.

- Case $E = \square$

  Choose $E^+ = E^* = \square$

- Case $E = E_1 e$

  $E_1 = E_1^+[E_1^*]$ — I.H.
  $E = E_1^+[E_1^*]\, e$ — Definition

  - Case $E_1^+ = \square$

    $E = E_1^*\, e$ — By the case assumption
    $E^+ = \square, E^* = E_1^*\, e$ — Choose

  - Case $E_1^+ = E_2^+\left[E_2^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$

    $E =$ — By the case assumption
    $E_2^+\left[E_2^*\left[\mathsf{check}_j^k(E_1^*, v)\right]\right]\, e = (E_2^+\, e)\left[E_2^*\left[\mathsf{check}_j^k(E_1^*, v)\right]\right]$
    $E^+ = (E_2^+\, e)\left[E_2^*\left[\mathsf{check}_j^k(\square, v)\right]\right], E^* = E_1^*$ — Choose

  Similarly to other basic cases.

- Case $E = \mathsf{check}_j^k(E_1, v)$

  $E_1 = E_1^+[E_1^*]$ — I.H.
  $E = \mathsf{check}_j^k(E_1^+[E_1^*], v) = \mathsf{check}_j^k(E_1^+, v)[E_1^*]$ — Definition

  - Case $E_1^+ = \square$

    $E = \mathsf{check}_j^k(\square, v)[E_1^*]$ — By the case assumption
    $E^+ = \mathsf{check}_j^k(\square, v), E^* = E_1^*$ — Let

  - Case $E_1^+ = E_2^+\left[E_2^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$.

    $E = \mathsf{check}_j^k\left(E_2^+\left[E_2^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right], v\right)[E_1^*]$ — By the case assumption
    $E^+ = \mathsf{check}_j^k\left(E_2^+\left[E_2^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right], v\right), E^* = E_1^*$ — Let; [Lemma E.5]

□

Lemma E.5. (Context composition preserves check structure)

If $E^+ = E_0^+\left[E_0^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$ and E is any evaluation context, then $E[E^+] = E_1^+$ for some E_1^+ of the form $E_1^+ = E_2^+\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]$.

Proof. By induction on the structure of E.

Base case: $E = \square$.

$E[E^+] = E^+ = E_0^+\left[E_0^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$ Required form

Inductive cases:

- Case $E = E_1\ e$. By I.H., $E_1[E^+] = E_1^+$ where $E_1^+ = E_2^+\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]$.

  $E[E^+] = E_1[E^+]\ e = E_2^+\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]\ e$ Definition

  $E[E^+] = (E_2^+\ e)\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]$ Reassociate context composition

  $E_3^+ = E_2^+\ e$ Take; required form

- Case $E = v\ E_1$. Similar to the previous case.

- Case $E = \mathsf{check}_{j_0}^{k_0}(E_1, v_0)$. By I.H., $E_1[E^+] = E_1^+$.

  $E[E^+] = \mathsf{check}_{j_0}^{k_0}(E_1^+, v_0)$ Definition

  If $E_1^+ = \square$, then $E[E^+] = \mathsf{check}_{j_0}^{k_0}(\square, v_0)$ which has the required form.

  If $E_1^+ = E_2^+\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]$, then:

  $E[E^+] = \mathsf{check}_{j_0}^{k_0}\left(E_2^+\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right], v_0\right)$ Case assumption

  $E[E^+] = \mathsf{check}_{j_0}^{k_0}(E_2^+, v_0)\left[E_2^*\left[\mathsf{check}_{j'}^{k'}(\square, v')\right]\right]$ Reassociate context composition

  $E_3^+ = \mathsf{check}_{j_0}^{k_0}(E_2^+, v_0)$ Take; required form

- Other cases (mon contexts) follow similarly.

In all cases, composing any context with a check-containing context yields another check-containing context. □

Lemma E.6. (Compiler is subset of relation)

If $\Gamma \vdash e_1 : \tau \rightarrowtail e_2$ then $e_1 \sim e_2^*$.

In particular, for closed programs, if $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then $e_1 \sim e_2^*$.

In this lemma, we use e^* as shorthand to denote e untyped.

Proof.

By induction on structure of $\Gamma \vdash e_1 : \tau \rightarrowtail e_2$.

- Case

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau \rightarrowtail x}$$

  $e_2^* = x$ Definition

  $x \sim x$ Relation

- Case

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \rightarrowtail e'}{\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau_2 \rightarrowtail \lambda x : \tau_1.e'}$$

$e_2^* = \lambda x.e'^*$ Definition

$e \sim e'^*$ I.H. on the premise with context $\Gamma, x : \tau_1$

$\lambda x.e \sim \lambda x.e'^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6 \rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^* \rangle\rangle$ Definition

$e_3 \sim e_5^*$ I.H.

$e_4 \sim e_6^*$ I.H.

$e_1 \sim e_2^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau_2 \rightarrowtail e_4 \qquad \Gamma \vdash \lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

$e_3 \sim e_4^*$ I.H.

$\lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\Big\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\Big\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

$e_3 \sim e_4^*$ I.H.

$\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\Big\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\Big\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$ Relation

- Other cases are similar.

□

E.3 Compiler Correctness

Figure 14 shows the structure of the proof. The source program e compiles to e', and the target program runs e' under wrap. After the handler is installed, the target reaches a state e_0'' with $\mathrm{EQ}(e, e_0'')$ (Lemma E.8). From then on, every source step $e_i \to e_{i+1}$ is matched by zero or more target steps $e_i'' \Longrightarrow^* e_{i+1}''$ that keep $\mathrm{EQ}(e_{i+1}, e_{i+1}'')$ (Lemma E.10). Chaining the squares (Lemma E.9), the source result r and the target result $\tilde{r}$ are related: a value goes to its related value, and blame_j^k goes to $\mathtt{error}_j$.

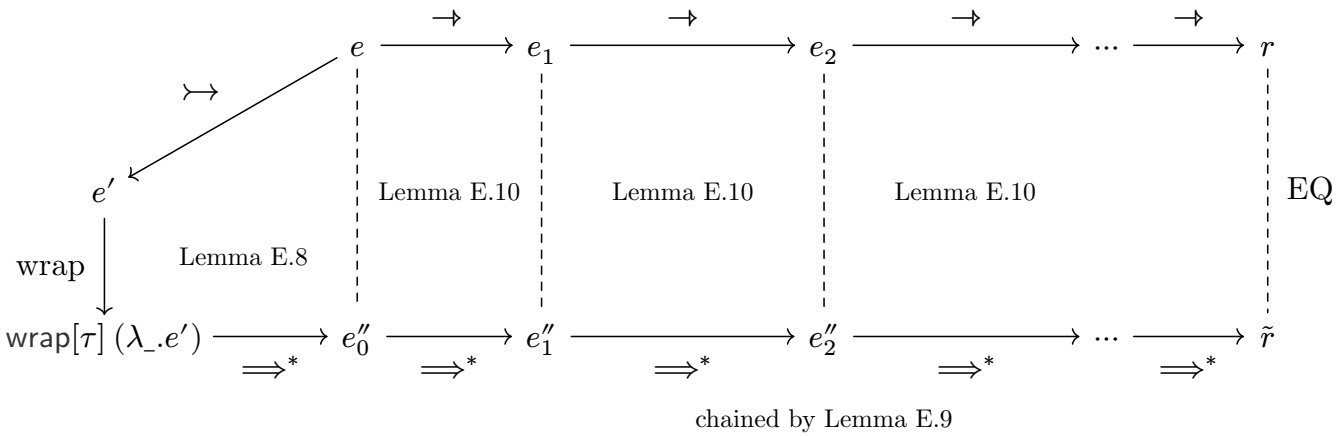


Figure 14: Structure of the compiler correctness proof. Dashed lines rel states by EQ.

Define a predicate between Contracts e and Untyped $\mathtt{Effects}$ e'.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e = v \text{ and } e' = \tilde{v} \\ \text{true if } e' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e} \text{ and } e \neq E^*\left[\mathsf{check}_j^k(e'', v)\right] \\ \text{true if } e = E^+\left[E^*\left[\mathsf{check}_j^k(e_1, v)\right]\right] \text{ and} \\ \qquad e' = \tilde{E}^+\left[\text{if } e_1' \text{ then } \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x]\right) \tilde{v} \text{ else } \mathtt{error}_j\right] \text{ and} \\ \qquad \mathrm{EQ}(e_1, e_1') \\ \text{true if } e = \mathsf{blame}_j^k \text{ and } e' = \mathtt{error}_j \\ \text{false otherwise} \end{cases}$$

Theorem E.7. (Compiler Correctness)

If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ and $e \to^* b$ then $e^* \Longrightarrow^* b$.

Proof.

$e^* \Longrightarrow^* e_1'$	Lemma E.8
$\mathrm{EQ}(e, e_1')$	Lemma E.8
$e_1' \Longrightarrow^* b$	Lemma E.9
$e^* \Longrightarrow^* b$	Combining steps

□

Lemma E.8. (Wrapped compiled program steps to an equivalent state with source program)

If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e_1^* = \text{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ then it exists e_2^* such that $e_1^* \Longrightarrow^* e_2^*$ and $\text{EQ}(e, e_2^*)$.

In this lemma, we use e^* as shorthand to denote e untyped.

Proof.

$e \sim e'^*$	Lemma E.6
$e_1^* \Longrightarrow^* \texttt{handler}\ \texttt{h}_{\mathscr{C}}\ \lambda_.e'^* \Longrightarrow^* \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ e'^* = e_2^*$	Wrapper reduction
$\text{EQ}(e, e_2^*)$	Definition

□

Lemma E.9. If $e \twoheadrightarrow^* r$ and $\text{EQ}(e, e')$ then $e' \Longrightarrow^* \tilde{r}$.

Proof. By breaking down the $\twoheadrightarrow^*$, and induction on it.

- Case $v \twoheadrightarrow^* v$

$e' = \tilde{r}$	Definition of $\text{EQ}(e, e')$
$e' \Longrightarrow^* \tilde{r}$	Reflexivity

- Case $\mathsf{blame}_j^k \twoheadrightarrow^* \mathsf{blame}_j^k$

$r = \mathsf{blame}_j^k$	Case assumption
$\tilde{r} = \texttt{error}_j$	Relation
$e' = \texttt{error}_j = \tilde{r}$	Definition of $\text{EQ}(e, e')$
$e' \Longrightarrow^* \tilde{r}$	Reflexivity

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

$e' \Longrightarrow^* e_1'$	Lemma E.10
$\text{EQ}(e_1, e_1')$	Lemma E.10
$e_1' \Longrightarrow^* \tilde{r}$	I.H.
$e' \Longrightarrow^* \tilde{r}$	Chaining steps

□

We also need to make a claim that in no where in e_1' do we have $\texttt{perform}\ \texttt{eff}$ that $\texttt{eff} \neq \mathscr{C}$.

Lemma E.10. If $e_1 \twoheadrightarrow e_2$ and $\text{EQ}(e_1, e_1')$ then there exists e_2' such that $e_1' \Longrightarrow^* e_2'$ and $\text{EQ}(e_2, e_2')$.

Proof of Lemma E.10. By case analysis on $e_1 \twoheadrightarrow e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$. By decomposition lemma, we know that the program can be rewritten into:

$e_1 = E[e_3]$ — Decomposition lemma
$e_2 = E[e_4]$ — Decomposition lemma
$e_3 \dashrightarrow e_4$ — Redex reduction

When $\mathrm{EQ}(e_1, e_1')$ holds by case 2, Lemma E.2 supplies the related check-free context $\tilde{E}^*$ occurring in e_1'. When it holds by case 3, the related contexts $\tilde{E}^+$ and $\tilde{E}^*$ are witnesses carried by the definition of EQ; we obtain them by inversion of that case.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) = e_4$
  - Case $e_1 = v$ is invalid.

    $E = E^+[E^*]$ — Context breakdown lemma
    $e_1 = E^+[E^*[e_3]]$ — Substitution
  - Case $E^+ = \square$

    By inversion of case 2 of $\mathrm{EQ}(e_1, e_1')$, $e_1 \sim \tilde{e_1}$. Since $e_1 = E^*[e_3]$, Lemma E.2 gives contexts and expressions that we name $\tilde{E}^*$ and $\tilde{e}_3$.

    $E^* \sim \tilde{E}^*$ — Lemma E.2
    $e_3 \sim \tilde{e}_3$ — Lemma E.2
    $\tilde{e_1} = \tilde{E}^*[\tilde{e}_3]$ — Lemma E.2
    $e_1 = E^*[e_3] \sim \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle \tilde{e}, \tilde{v} \rangle]$ — Relation
    $e_1' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle \tilde{e}, \tilde{v} \rangle]$ — $\mathrm{EQ}(e_1, e_1')$, case 2
    $e_1' \Longrightarrow^*$ — Reduction
    $\text{if } \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e}\ \tilde{v} \text{ then } (\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x])\ \tilde{v} \text{ else } \mathtt{error}_j =$
    e_2'
    $\mathrm{EQ}(e\ v, \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e}\ \tilde{v})$ — Case 2
    $\mathrm{EQ}(e_2, e_2')$ — Case 3
  - Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

    $e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(E^*[e_3], v_1)\right]\right]$ — Context decomposition
    $e_1' = \tilde{E}_1^+\left[\text{if } \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{e}_3] \text{ then } (\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}_1^*[x])\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\right]$ — Definition of EQ
    $e_1' = \tilde{E}_1^+\left[\text{if } \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle \tilde{e}, \tilde{v} \rangle] \text{ then } (\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}_1^*[x])\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\right]$ — Relation
    $e_1' \Longrightarrow^* \tilde{E}^+\left[\text{if } \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e}\ \tilde{v} \text{ then } (\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x])\ \tilde{v} \text{ else } \mathtt{error}_j\right] =$ — Reduction
    e_2'

    where $\tilde{E}^+ = \tilde{E}_1^+\left[\text{if } \square \text{ then } (\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}_1^*[x])\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(e\ v, v)\right], v_1\right)\right]\right]$ Reduction under context

$\mathrm{EQ}\left(E^*\left[\mathsf{check}_j^k(e\ v, v)\right], \mathtt{if}\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e}\ \tilde{v}\ \mathtt{then}\ \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x]\right)\tilde{v}\ \mathtt{else}\ \mathtt{error}_j\right)$ Case 3

$\mathrm{EQ}(e_2, e_2')$ Case 3

(Case 2 is invalid, because of the shape of e_2)

- Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$
  - Case $e_1 = v$ is invalid.
  - Case $e_1 = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]\right]$.

    $e_1' = \tilde{E}^+\left[\mathtt{if}\ e_3'\ \mathtt{then}\ \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x]\right)\tilde{v}\ \mathtt{else}\ \mathtt{error}_j\right]$ $\mathrm{EQ}(e_1, e_1')$

    $\mathrm{EQ}(\mathsf{true}, e_3')$ Since true is a value

    $e_3' = \mathsf{true}$ Definition of EQ

    $e_1' \Longrightarrow^* \tilde{E}^+\left[\mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{v}]\right]$ Reduction

    $e_2 = E^+[E^*[v]]$ Source reduction

    – Case $E^+ = \square$

      $\mathrm{EQ}\left(E^*[v], \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{v}]\right)$ Case 2

    – Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

      $\tilde{E}^+ = \tilde{E}_1^+\left[\mathtt{if}\ \square\ \mathtt{then}\ \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}_1^*[x]\right)\tilde{v}_1\ \mathtt{else}\ \mathtt{error}_{j_1}\right]$ Relation

      $e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(E^*[v], v_1)\right]\right]$ Source reduction

      $e_2' = \tilde{E}_1^+\left[\mathtt{if}\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{v}]\ \mathtt{then}\ \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}_1^*[x]\right)\tilde{v}_1\ \mathtt{else}\ \mathtt{error}_{j_1}\right]$ Target reduction

      $\mathrm{EQ}\left(E^*[v], \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{v}]\right)$ Case 2

      $\mathrm{EQ}(e_2, e_2')$ Case 3

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) = e_4$
  - Case $e_1 = v$ is invalid.

    $E = E^+[E^*]$ Context breakdown lemma

    $e_1 = E^+[E^*[e_3]]$ Substitution

  - Case $E^+ = \square$

    Let $e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) = \lambda f.\lambda x.e_{5a}$

    By inversion of case 2 of $\mathrm{EQ}(e_1, e_1')$, $e_1 \sim \tilde{e_1}$. Since $e_1 = E^*[e_3]$, Lemma E.2 gives contexts and expressions that we name $\tilde{E}^*$ and $\tilde{e}_3$.

    $E^* \sim \tilde{E}^*$ Lemma E.2

    $e_3 \sim \tilde{e}_3$ Lemma E.2

    $\tilde{e_1} = \tilde{E}^*[\tilde{e}_3]$ Lemma E.2

    $e_1 = E^*[e_3] \sim \tilde{E}^*[(\lambda f.\lambda x.\tilde{e}_{5a})\ \tilde{v}]$ Relation

$e_1' = \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[(\lambda f.\lambda x.\tilde{e}_{5a})\ \tilde{v}]$ — $\mathrm{EQ}(e_1, e_1')$, case 2

$e_1' \Longrightarrow^* \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]] = e_2'$ — Reduction

$e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$ — Source reduction

$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \lambda x.\tilde{e}_{5a}[f := \tilde{v}]$ — Relation on values

$\mathrm{EQ}(e_2, e_2')$ — Case 2

- Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

  Let $e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

  where $e_{5a} = \mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$.

  $e_1' = \tilde{E}_1^+\left[\text{if } \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{e}_5\ \tilde{v}] \text{ then } \left(\lambda x.\ \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}_1^*[x]\right)\ \tilde{v}_1 \text{ else } \texttt{error}_{j_1}\right]$ — Definition of EQ

  $e_1' \Longrightarrow^* \tilde{E}_1^+\left[\text{if } \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\ \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}_1^*[x]\right)\ \tilde{v}_1 \text{ else } \texttt{error}_{j_1}\right] = e_2'$ — Reduction

  $e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right], v_1\right)\right]\right]$ — Source reduction

  $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \lambda x.\tilde{e}_{5a}[f := \tilde{v}]$ — Relation on values

  $\mathrm{EQ}(e_2, e_2')$ — Case 3

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$

  - Case $e_1 = v$ is invalid.

    $E = E^+[E^*]$ — Context breakdown lemma

    $e_1 = E^+[E^*[e_3]]$ — Substitution

  - Case $E^+ = \square$

    where $e_5 = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$.

    By inversion of case 2 of $\mathrm{EQ}(e_1, e_1')$, $e_1 \sim \tilde{e_1}$. Since $e_1 = E^*[e_3]$, Lemma E.2 gives contexts and expressions that we name $\tilde{E}^*$ and $\tilde{e}_3$.

    $E^* \sim \tilde{E}^*$ — Lemma E.2

    $e_3 \sim \tilde{e}_3$ — Lemma E.2

    $\tilde{e_1} = \tilde{E}^*[\tilde{e}_3]$ — Lemma E.2

    $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ — Relation

    $e_1' = \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{e}_3]$ — $\mathrm{EQ}(e_1, e_1')$, case 2

    $e_1' \Longrightarrow^* \texttt{handle}\ \mathsf{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{e}_4] = e_2'$ — β

    $e_2 = E^*[e_4]$ — Source reduction

    $\mathrm{EQ}(e_2, e_2')$ — Case 2

  - Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

    Same as the case $E^+ = \square$, inside $\tilde{E}^+$, by case 3.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) = e_4$

Same as the case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)$, with

$e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\left\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\right\}\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) =$ Definition
$\lambda f.\lambda x.e_{5a}$

$\lambda x.\tilde{e}_{5a}[f := \tilde{v}] \sim \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)$ Relation on values

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$ where $\kappa_3 = \left\{\mathsf{mon}_j^{l,j}(\kappa_1, v_2)/y\right\}\kappa_2$
  - Case $e_1 = v$ is invalid.

  Context breakdown lemma, $E = E^+[E^*]$.

  Let $e_6 = \mathsf{mon}_j^{k,l}\left(\left\{\mathsf{mon}_j^{l,j}(\kappa_1, x)/y\right\}\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$. Since $x \notin \mathrm{fv}(v_1, \kappa_1, \kappa_2)$, $e_4 = e_6[x := v_2]$.

  - Case $E^+ = \square$

    By inversion of case 2 of $\mathrm{EQ}(e_1, e_1')$, $e_1 \sim \tilde{e_1}$. Since $e_1 = E^*[e_3]$, Lemma E.2 gives contexts and expressions that we name $\tilde{E}^*$ and $\tilde{e}_3$.

    $E^* \sim \tilde{E}^*$ Lemma E.2
    $e_3 \sim \tilde{e}_3$ Lemma E.2
    $\tilde{e_1} = \tilde{E}^*[\tilde{e}_3]$ Lemma E.2
    $\tilde{e}_3 = (\lambda x.\tilde{e}_6)\ \tilde{v}_2$ The `guard` row of the relation on values
    $e_1' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[(\lambda x.\tilde{e}_6)\ \tilde{v}_2]$ $\mathrm{EQ}(e_1, e_1')$, case 2
    $e_1' \Longrightarrow^* \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[\tilde{e}_6[x := \tilde{v}_2]] = e_2'$ β
    $e_2 = E^*[e_6[x := v_2]]$ Source reduction
    $e_6[x := v_2] \sim \tilde{e}_6[x := \tilde{v}_2]$ Lemma E.3
    $\mathrm{EQ}(e_2, e_2')$ Case 2

  - Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

    Same as the case $E^+ = \square$, inside $\tilde{E}^+$, by case 3.

  The indy monitors $\mathsf{mon}_j^{l,j}(\kappa_1, v_2)$ inside κ_3 are ordinary `mon` redexes. Each one is related to the compiled monitor at the same position in $\tilde{e}_6[x := \tilde{v}_2]$, so when the source evaluates one, it is one of the `mon` cases above.

- Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$
  - Case $e_1 = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$

    $e_1' = \tilde{E}^+\left[\mathsf{if\ false\ then}\ \left(\lambda x.\ \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}^*[x]\right)\ \tilde{v}\ \mathsf{else}\ \mathtt{error}_j\right]$ $\mathrm{EQ}(e_1, e_1')$
    $e_1' \Longrightarrow^* \tilde{E}^+\left[\mathtt{error}_j\right] = e_2'$ Reduction
    $e_2 = E^+\left[E^*\left[\mathsf{blame}_j^k\right]\right]$ Source reduction
    $\tilde{E}^+\left[\mathtt{error}_j\right] \Longrightarrow \mathtt{error}_j$ `Step-E-Error`

$\mathsf{blame}_j^k \sim \mathtt{error}_j$ Relation

$\mathrm{EQ}(e_2, e_2')$ Definition

□

CHAPTER F

Compiler Correctness Proof (Shallow Handler)

F.1 Overview

Proof for compiler from Contracts to Untyped Shallow Effects, the handler semantic is shallow, where the continuation does not reinstall the handler.

The only rule that changes from the deep semantics is Step-E-Perform, the continuation k no longer wraps E with $\texttt{handle}\ h\ \cdot$.

$$\frac{\mathsf{op} \notin \texttt{bop}(E) \wedge (\mathsf{op} \to f) \in h \qquad k = \lambda x.E[x]}{\texttt{handle}\ h\ E[\texttt{perform}\ \mathsf{op}\ v] \quad \rightarrow \quad f\ v\ k}\ \text{Step-E-Perform-Shallow}$$

$$\texttt{h}_{\mathscr{C}} : \texttt{lab} \quad = \quad \{\mathscr{C} \to \forall\alpha.\langle \texttt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\}$$

$$\mathsf{H} \quad = \quad \mu h.\ \texttt{handler}\ \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.h\ (\lambda_.\ \text{if}\ e\ v\ \text{then}\ k\ v\ \text{else}\ \texttt{error}_l)\}$$

$$\mathsf{wrap} \quad = \quad \Lambda\alpha.\ \lambda f : \mathsf{unit} \to \alpha.\ \mathsf{H}\ f$$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the argument as tuple pattern

The handler clause re-installs H around the rest of the program by itself, instead of the continuation doing it. So the whole program always runs under exactly one $\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \cdot$ at the top, and the predicate of a check runs under the same handler as the program. We write $\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ e$ for the program running under the handler of H.

Compared to the deep semantics, there are no states without handle, and there is no nested handle for predicates. This makes the relation much simpler.

Compiler Correctness by proving (1) Contracts $\sim$ Untyped Effects, (2) Untyped Effects $\sim$ Effects, then obtain Contracts $\sim$ Effects. (2) is trivial.

In this proof, we use e^* as shorthand to denote e untyped.

Assumptions:

- Contract predicates are values ($\mathsf{flat}(e)$ with $e = \lambda x.e_0$) and pure (they do not use the store).
- Tuple contracts and mutable cells are not covered.

F.2 Relation

Let's first define these relations:

- $R_e \subseteq$ Contracts $e \times$ Untyped $\texttt{Effects}\ e$

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $E \times$ Untyped $\texttt{Effects}\ E$

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

In the shallow semantics, the innermost check captures everything outside of it as its continuation, so a check is related together with its whole context:

$E\left[\mathsf{check}^k_j(e, v)\right] \sim \text{if } \tilde{e} \text{ then } \left(\lambda x.\widetilde{E[x]}\right) \tilde{v} \text{ else } \texttt{error}_k$

where e has no check in evaluation position. E may contain other checks, $\widetilde{E[x]}$ is again related by this rule, and it has one less check.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e' = \texttt{handle h}_{\mathscr{C}}\ \tilde{e} \\ \text{true if } e = E\left[\mathsf{blame}^k_j\right] \text{ and } e' = \texttt{handle h}_{\mathscr{C}}\ \texttt{error}_k \\ \text{false otherwise} \end{cases}$$

$\sim \subseteq$ Contracts $E \times$ Untyped $\texttt{Effects}\ E$

Define $\tilde{E}$ such that $E \sim \tilde{E}$. If E contains a check, only the last row applies, on the innermost check.

$$\begin{array}{rcl} \square & \sim & \square \\ E\ e & \sim & \tilde{E}\ \tilde{e} \\ v\ E & \sim & \tilde{v}\ \tilde{E} \\ \mathsf{mon}^{k,l}_j(\mathsf{flat}(e), E) & \sim & \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{E} \rangle \rangle \end{array}$$

$$
\begin{array}{rcl}
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E) & \sim & \tilde{e}_1\ \tilde{E} \\
 & & e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) \text{ row below} \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E) & \sim & \tilde{e}_1\ \tilde{E} \\
 & & e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e) \text{ row below} \\
E\big[\mathsf{check}_j^k(E_1, v)\big] & \sim & \text{if } \tilde{E}_1 \text{ then } \Big(\lambda x.\widetilde{E[x]}\Big)\ \tilde{v} \text{ else } \mathtt{error}_k \\
 & & \text{where } E_1 \text{ has no check}
\end{array}
$$

The other contexts ($\langle E, e\rangle$, $o(v..., E, e...)$, if E then e_1 else e_2, ...) are related homomorphically.

$\sim \subseteq$ Contracts $e \times$ Untyped `Effects` e

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2\rangle\rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i & \sim & \tilde{e}_1\ \tilde{e} \\
\lambda y.\kappa_2, e) & & \\
 & & e_1 = \lambda f.\lambda x.\Big(\lambda y.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)\Big)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \\
E\big[\mathsf{check}_j^k(e, v)\big] & \sim & \text{if } \tilde{e} \text{ then } \Big(\lambda x.\widetilde{E[x]}\Big)\ \tilde{v} \text{ else } \mathtt{error}_k \\
 & & \text{where } e \text{ has no check in evaluation position (innermost check)} \\
\{e_y/y\}e & \sim & \tilde{e}\big[y := \tilde{v}_y\big] \\
 & & \text{where } e_y = \mathsf{mon}_j^{l,j}(\kappa_1, v),\ e_y \twoheadrightarrow^* v_y \text{ and } v_y \sim \tilde{v}_y \text{ (indy argument)} \\
\mathsf{blame}_j^k & \sim & \mathtt{error}_k
\end{array}
$$

The other expressions ($\langle e_1, e_2\rangle$, $o(e...)$, if e_1 then e_2 else e_3, $\mu x.e$, ...) are related homomorphically.

$\sim \subseteq$ Contracts $v \times$ Untyped `Effects` v

$$
\begin{array}{rcl}
b & \sim & b \\
N & \sim & N \\
\lambda x.e & \sim & \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x)
\end{array}
$$

Lemma F.1. (Relation is correct) If $E \sim \tilde{E}$, $e \sim \tilde{e}$ and e has no check in evaluation position, then $E[e] \sim \tilde{E}[\tilde{e}]$.

Proof. By case on E.

- Case E has no check

  By induction on E, every row of $E \sim \tilde{E}$ matches a row of $e \sim \tilde{e}$ with the hole filled.

- Case $E = E_0\left[\mathsf{check}_j^k(E_1, v)\right]$ where E_1 has no check

  $E[e] = E_0\left[\mathsf{check}_j^k(E_1[e], v)\right]$ and $E_1[e]$ has no check in evaluation position.

  $E[e] \sim$ if $\widetilde{E_1[e]}$ then $\left(\lambda x.\widetilde{E_0[x]}\right)\ \tilde{v}$ else $\mathtt{error}_k$ by the check row.

  $\widetilde{E_1[e]} = \tilde{E}_1[\tilde{e}]$ by the previous case.

  $= \tilde{E}[\tilde{e}]$ by the check row of $E \sim \tilde{E}$.

□

Lemma F.2. (Relation is closed under substitution) If $e \sim \tilde{e}$ and $v \sim \tilde{v}$ then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$.

Proof. By induction on $e \sim \tilde{e}$. Every row is homomorphic on x, and the bound variables f, x, y in the rows are chosen fresh. □

Lemma F.3. (Compiler is subset of relation)

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then $e_1 \sim e_2^*$.

Proof.

By induction on structure of $\cdot \vdash e_1 : \tau \rightarrowtail e_2$.

- Case

$$\frac{\cdot \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail e_5 \qquad \cdot \vdash e_4 : \tau \rightarrowtail e_6}{\cdot \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6\rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^*\rangle\rangle$

By I.H. $e_3 \sim e_5^*$ and $e_4 \sim e_6^*$.

Then $e_1 \sim e_2^*$ by relation.

- Case

$$\frac{\cdot \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail e_4 \qquad \cdot \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5}{\cdot \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

By I.H., $e_3 \sim e_4^*$

By I.H., $\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \sim e_5^*$

Then $e_1 \sim e_2^*$ by relation.

- Case

$$\frac{\cdot \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail e_4 \qquad \cdot \vdash \lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5}{\cdot \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

By I.H., $e_3 \sim e_4^*$

By I.H., $\lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \sim e_5^*$

Then $e_1 \sim e_2^*$ by relation.

- Other cases are homomorphic, by I.H.

□

F.3 Compiler Correctness

Theorem F.4. If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ and $e \twoheadrightarrow^* r$ then $e^* \Longrightarrow^* \tilde{r}$.

Where r is a value v or blame_j^k.

Proof.

$e^* \Longrightarrow^* e_1'$ and $\mathrm{EQ}(e, e_1')$ by Lemma F.5.

$e_1' \Longrightarrow^* \tilde{r}$ by Lemma F.6.

$e^* \Longrightarrow^* \tilde{r}$ by combining steps. □

Lemma F.5. (Wrapped compiled program steps to an equivalent state with source program)

If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e_1^* = \text{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ then it exists e_2^* such that $e_1^* \Longrightarrow^* e_2^*$ and $\text{EQ}(e, e_2^*)$.

Proof.

By Lemma F.3, $e \sim e'^*$.

$e_1^* \Longrightarrow^* \mathsf{H}\ \lambda_.e'^* \Longrightarrow^* \texttt{handler}\ \mathtt{h}_{\mathscr{C}}\ \lambda_.e'^* \Longrightarrow^* \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ e'^* = e_2^*$

By definition, $\text{EQ}(e, e_2^*)$ (case 1). □

Lemma F.6. If $e \twoheadrightarrow^* r$ and $\text{EQ}(e, e')$ then $e' \Longrightarrow^* \tilde{r}$.

Proof. By breaking down the $\twoheadrightarrow^*$, and induction on it.

- Case $r \twoheadrightarrow^* r$ with $r = v$

  $e' = \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{v}$ because $\text{EQ}(v, e')$ (case 2 is invalid).

  $e' \blacktriangleright \tilde{v}$ by Step-E-Return.

- Case $r \twoheadrightarrow^* r$ with $r = \mathsf{blame}_j^k$

  $e' = \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \texttt{error}_k$ in both cases of `treq`.

  $e' \Longrightarrow \texttt{error}_k$ by Step-E-Error.

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

  By simulation, we have $e' \Longrightarrow^* e_1'$ such that $\text{EQ}(e_1, e_1')$.

  By IH, $e_1' \Longrightarrow^* \tilde{r}$.

  By chaining steps, $e' \Longrightarrow^* \tilde{r}$.

□

We also need to make a claim that in no where in $\tilde{e}$ do we have `handle` or $\texttt{perform}\ \texttt{eff}$ that $\texttt{eff} \neq \mathscr{C}$, so $\mathscr{C} \notin \texttt{bop}(E')$ for every target context E' under the top handle.

Lemma F.7. (Performing a check) If E' has no `handle`, then

$$\texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ E'[\texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v} \rangle\rangle]$$

$$\Longrightarrow^* \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \text{ if } \tilde{e}\ \tilde{v} \text{ then } (\lambda x.E'[x])\ \tilde{v} \text{ else } \texttt{error}_k$$

Proof.

$\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ E'[\texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v} \rangle\rangle]$

$\rightarrow (\lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \mathsf{H}\ (\lambda_.\ \text{if}\ e\ v\ \text{then}\ k\ v\ \text{else}\ \texttt{error}_l))\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle\ (\lambda x.E'[x])$ (Step-E-Perform-Shallow)

$\Longrightarrow^* \mathsf{H}\ (\lambda_.\ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ (\lambda x.E'[x])\ \tilde{v}\ \text{else}\ \texttt{error}_k)$

$\Longrightarrow^* \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ (\lambda x.E'[x])\ \tilde{v}\ \text{else}\ \texttt{error}_k$ (unfold μ, Step-E-Handler, β)

□

Lemma F.8. If $e_1 \twoheadrightarrow e_2$ and $\mathrm{EQ}(e_1, e_1')$ then there exists e_2' such that $e_1' \Longrightarrow^* e_2'$ and $\mathrm{EQ}(e_2, e_2')$.

Proof of Lemma F.8. By case analysis on $e_1 \twoheadrightarrow e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$.

- Case $e_1 = E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k = e_2$ (Step-C-Blame)
  - Case $\mathrm{EQ}(e_1, e_1')$ is case 2, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \texttt{error}_k$.

    $e_2' = e_1'$, and $\mathrm{EQ}\left(\mathsf{blame}_j^k, e_2'\right)$ (case 2 with $E = \square$).
  - Case $\mathrm{EQ}(e_1, e_1')$ is case 1, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \tilde{E}[\texttt{error}_k]$.

    $e_1' \Longrightarrow \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \texttt{error}_k = e_2'$ by Step-E-Error.

    Obtain $\mathrm{EQ}\left(\mathsf{blame}_j^k, e_2'\right)$ (case 2 with $E = \square$).

Otherwise, e_1 is not blamed and $\mathrm{EQ}(e_1, e_1')$ is case 1. By decomposition lemma, $e_1 = E[e_3]$, $e_2 = E[e_4]$ and that $e_3 \dashrightarrow e_4$ is a redex reduction.

If e_3 is not a check, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_3]$ by Lemma F.1.

Note that $\tilde{E}$ is an evaluation context of `Effects` without `handle`. Unlike the deep proof, there is no case split on the checks in E, the redex steps under the same top handle.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) = e_4$

  $\tilde{e}_3 = \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle$

  $e_1' \Longrightarrow^* \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ \left(\lambda x.\tilde{E}[x]\right)\ \tilde{v}\ \text{else}\ \texttt{error}_k = e_2'$ by Lemma F.7.

  $e_2 = E\left[\mathsf{check}_j^k(e\ v, v)\right]$, and $e\ v$ has no check in evaluation position.

  $e_2 \sim \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ \left(\lambda x.\widetilde{E[x]}\right)\ \tilde{v}\ \text{else}\ \texttt{error}_k$ by the check row.

  $\widetilde{E[x]} = \tilde{E}[x]$ by Lemma F.1.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1).
- Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$

$e_1 = E\big[\mathsf{check}^k_j(\mathsf{true}, v)\big]$, the check is the innermost one.

$e'_1 = \texttt{handle h}_{\mathscr{C}}$ if true then $\Big(\lambda x.\widetilde{E[x]}\Big)\ \tilde{v}$ else $\texttt{error}_k$ by the check row.

$e'_1 \Longrightarrow^* \texttt{handle h}_{\mathscr{C}}\ \widetilde{E[x]}[x := \tilde{v}] = e'_2$

$e_2 = E[v] = E[x][x := v] \sim \widetilde{E[x]}[x := \tilde{v}]$ by Lemma F.2.

Obtain $\mathrm{EQ}(e_2, e'_2)$ (case 1).

- Case $e_3 = \mathsf{check}^k_j(\mathsf{false}, v) \dashrightarrow \mathsf{blame}^k_j = e_4$

  $e_1 = E\big[\mathsf{check}^k_j(\mathsf{false}, v)\big]$, the check is the innermost one.

  $e'_1 = \texttt{handle h}_{\mathscr{C}}$ if false then $\Big(\lambda x.\widetilde{E[x]}\Big)\ \tilde{v}$ else $\texttt{error}_k$ by the check row.

  $e'_1 \Longrightarrow \texttt{handle h}_{\mathscr{C}}\ \texttt{error}_k = e'_2$

  $e_2 = E\big[\mathsf{blame}^k_j\big]$

  Obtain $\mathrm{EQ}(e_2, e'_2)$ (case 2).

- Case $e_3 = \mathsf{mon}^{k,l}_j(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v) = e_4$

  Let $e_5 = \lambda f.\lambda x.\mathsf{mon}^{k,l}_j\Big(\kappa_2, f\ \mathsf{mon}^{l,k}_j(\kappa_1, x)\Big) = \lambda f.\lambda x.e_{5a}$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}$

  $e'_1 \Longrightarrow \texttt{handle h}_{\mathscr{C}}\ \tilde{E}[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]] = e'_2$ by β.

  $\mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v) \sim \lambda x.\tilde{e}_{5a}[f := \tilde{v}]$ by the relation on values.

  Obtain $\mathrm{EQ}(e_2, e'_2)$ (case 1) by Lemma F.1.

- Case $e_3 = \mathsf{mon}^{k,l}_j(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}^{k,l}_j(\kappa_1 \to_i \lambda y.\kappa_2, v) = e_4$

  Same as the case $\mathsf{mon}^{k,l}_j(\kappa_1 \to \kappa_2, v)$, with

  $e_5 = \lambda f.\lambda x.\Big(\lambda y.\mathsf{mon}^{k,l}_j\Big(\kappa_2, f\ \mathsf{mon}^{l,k}_j(\kappa_1, x)\Big)\Big)\ \mathsf{mon}^{l,j}_j(\kappa_1, x) = \lambda f.\lambda x.e_{5a}$

- Case $e_3 = \mathsf{guard}^{k,l}_j(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}^{k,l}_j\Big(\kappa_2, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\Big) = e_4$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ where $e_5 = \lambda x.\mathsf{mon}^{k,l}_j\Big(\kappa_2, \tilde{v}_1\ \mathsf{mon}^{l,k}_j(\kappa_1, x)\Big)$

  $e'_1 \Longrightarrow \texttt{handle h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_4] = e'_2$ by β and Lemma F.2.

  Obtain $\mathrm{EQ}(e_2, e'_2)$ (case 1) by Lemma F.1.

- Case $e_3 = \mathsf{guard}^{k,l}_j(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}^{k,l}_j\Big(\kappa_3, v_1\ \mathsf{mon}^{l,k}_j(\kappa_1, v_2)\Big) = e_4$ where $\kappa_3 = \Big\{\mathsf{mon}^{l,j}_j(\kappa_1, v_2)/y\Big\}\kappa_2$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ where $e_5 = \lambda x.(\lambda y.e_{5a})\ \mathsf{mon}^{l,j}_j(\kappa_1, x)$ and $e_{5a} = \mathsf{mon}^{k,l}_j\Big(\kappa_2, \tilde{v}_1\ \mathsf{mon}^{l,k}_j(\kappa_1, x)\Big)$

  $e'_1 \Longrightarrow \texttt{handle h}_{\mathscr{C}}\ \tilde{E}[(\lambda y.\tilde{e}_{5a}[x := \tilde{v}_2])\ \tilde{e}_7]$ by β, where $e_7 = \mathsf{mon}^{l,j}_j(\kappa_1, v_2)$

The target checks the indy argument $\mathsf{mon}_j^{l,j}(\kappa_1, v_2)$ first, the source checks the argument $\mathsf{mon}_j^{l,k}(\kappa_1, v_2)$ first. For $\kappa_1 = \mathsf{flat}(e_p)$ both become $\mathsf{check}_j^l(e_p\, v_2, v_2)$, so we pair the two checks.

- Case the check fails.

  Both programs reach $\mathsf{check}_j^l(\mathsf{false}, v_2)$, then blame_j^l and $\texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \texttt{error}_l$ as in the case $\mathsf{check}_j^k(\mathsf{false}, v)$.

- Case the check succeeds, $\mathsf{mon}_j^{l,j}(\kappa_1, v_2) \twoheadrightarrow^* v_y$ ($v_y = v_2$ for flat κ_1, otherwise $v_y = \mathsf{guard}_j^{l,j}(\kappa_1, v_2)$).

  Let $e_6 = \mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$, then $e_2 = E\big[\big\{\mathsf{mon}_j^{l,j}(\kappa_1, v_2)/y\big\}e_6\big]$.

  $e_1' \Longrightarrow^* \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}\big[\tilde{e}_6\big[y := \tilde{v}_y\big]\big] = e_2'$

  By purity, the target's argument check $\mathsf{mon}_j^{l,k}(\kappa_1, \tilde{v}_2)$ succeeds again. Each later evaluation of $\mathsf{mon}_j^{l,j}(\kappa_1, v_2)$ inside κ_2 in the source gives v_y again and is matched by zero target steps.

  $\big\{\mathsf{mon}_j^{l,j}(\kappa_1, v_2)/y\big\}e_6 \sim \tilde{e}_6\big[y := \tilde{v}_y\big]$ by the indy row.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1).

- Case e_3 is a base reduction (β, μ, `injl`, `injr`, `ife`, o)

  $\tilde{e}_3$ is the same redex in `Effects`, and $\tilde{e}_3 \twoheadrightarrow \tilde{e}_4$ by the same rule, using Lemma F.2 for β and μ.

  $e_1' \Longrightarrow \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_4] = e_2'$

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1) by Lemma F.1.

□